A Survey on Retrieval-Augmented Text Generation for Large Language Models

YIZHENG HUANG, Information Retrieval and Knowledge Management Research Lab, York University, Toronto, Canada
JIMMY XIANGJI HUANG, Information Retrieval and Knowledge Management Research Lab, York University, Toronto, Canada

Retrieval-Augmented Generation (RAG) merges information retrieval (IR) techniques with deep learning advancements to address the static limitations of large language models (LLMs) by enabling the dynamic integration of up-to-date external information. This methodology, focusing primarily on the text domain, provides a cost-effective solution to the generation of plausible but possibly incorrect responses by LLMs, thereby enhancing the accuracy and reliability of their outputs through the use of real-world data. As RAG grows in complexity and incorporates multiple concepts that can influence its performance, this article organizes the RAG paradigm into four categories: pre-retrieval, retrieval, post-retrieval, and generation, offering a detailed perspective from the retrieval viewpoint. It outlines RAG's mechanics and discusses the field's progression through the analysis of significant studies. Additionally, the article introduces evaluation methods for RAG, addressing the challenges faced and proposing future research directions. By offering an organized framework and categorization, the study aims at consolidating existing research on RAG, clarify its technological underpinnings, and highlight its potential to broaden the adaptability and applications of LLMs.

CCS Concepts: • **Computing methodologies** → **Artificial intelligence**; • **Information systems** → **Information retrieval;**

Additional Key Words and Phrases: RAG, retrieval-augmented generation, IR, information retrieval, LLM, large language model, generative information retrieval, NLP, natural language processing, QA, question answering, chatbot, ChatGPT, hallucination

ACM Reference Format:
Yizheng Huang and Jimmy Xiangji Huang. 2026. A Survey on Retrieval-Augmented Text Generation for Large Language Models. *ACM Comput. Surv.* 58, 12, Article 300 (May 2026), 38 pages. https://doi.org/10.1145/3805774

This research is supported by the research grant (RGPIN-2020-07157) from the Natural Sciences and Engineering Research Council (NSERC) of Canada and the York Research Chairs (YRC) program.
Authors' Contact Information: Yizheng Huang, Information Retrieval and Knowledge Management Research Lab, York University, Toronto, Ontario, Canada; e-mail: hyz@yorku.ca; Jimmy Xiangji Huang (corresponding author), Information Retrieval and Knowledge Management Research Lab, York University, Toronto, Ontario, Canada; e-mail: jhuang@yorku.ca.



This work is licensed under a Creative Commons Attribution 4.0 International License.

© 2026 Copyright held by the owner/author(s).
ACM 0360-0300/2026/05-ART300
https://doi.org/10.1145/3805774

1 Introduction

The advent of ChatGPT has significantly impacted both academia and industry due to its interactive capabilities and widespread application, establishing itself as a leading artificial intelligence tool. At the core of ChatGPT is the GPT series of large language models, as detailed by [1], which has seen substantial improvements over its predecessors, showcasing exceptional abilities in a variety of **Natural Language Processing** (**NLP**) tasks [25, 87, 88]. Despite these advancements, the adoption of LLMs has highlighted several critical issues mainly due to their reliance on extensive datasets. This reliance restricts their ability to incorporate new information after training, leading to three primary challenges [62, 67, 86, 192]. First, the focus on broad and general data to maximize accessibility and applicability results in subpar performance in specialized areas. Second, the rapid creation of online data, combined with the significant resources required for data annotation and model training, hinders LLMs' ability to stay updated. Third, LLMs are susceptible to generating convincing yet inaccurate responses, known as "hallucinations", which can mislead users.

Addressing these challenges is crucial for LLMs to be effectively utilized across various domains. A promising solution is the integration of **Retrieval-Augmented Generation** (**RAG**) technology, which supplements models by fetching external data in response to queries, thus ensuring more accurate and current outputs. Figure 1 illustrates how RAG can enable ChatGPT to provide precise answers beyond its initial training data.

Since Lewis et al. [91] introduced RAG in 2020, its development has been rapid, especially with the growth of models like ChatGPT. This rapid growth has resulted in research that is often disconnected and uses different names for similar methods, which can be confusing. The situation is further complicated by the fact that a single proposed method can affect multiple stages of the RAG pipeline, making it difficult to identify its main contribution. To address these challenges, this survey introduces a unified paradigm that organizes existing approaches into four distinct categories from an **information retrieval** (**IR**) perspective: pre-retrieval, retrieval, post-retrieval, and generation. This framework allows us to precisely identify which components are being enhanced, thereby helping to organize the research topics and clarify the purpose of different methods. This survey focuses primarily on textual applications, which have been the most prominent in recent research.

To construct this framework and analyze the techniques within each phase, we selected studies through a careful review of the existing literature. Our selection process focused on original and important studies that introduced key concepts, are highly cited, and were published in top-tier conferences and journals in the fields of IR, NLP, and AI. We also included important pre-prints that present new ideas or have already shown a considerable impact on the field. This methodology ensures that our survey covers the techniques that have most significantly shaped the development and discussion of RAG.

Other related surveys provide valuable insights into the evolving RAG landscape from different angles. For instance, the work by Gao et al. [43] organizes the field into three development paradigms: Naive, Advanced, and Modular RAG. While this provides a useful high-level view of the system's complexity, it does not detail the specific steps within the RAG process from an information retrieval standpoint. The survey by Zhao et al. [187] reviews a wide range of applications across different types of content, including text, images, and video. Due to its broad focus on multiple modalities, it does not offer a deep analysis of the techniques specific to text generation. Similarly, the study by Hu et al. [54] classifies Retrieval-Augmented Language Models by the interaction patterns between the retriever and the language model, covering both generation and understanding tasks. Their work focuses on the architectural flow of information but does not examine the RAG pipeline through the distinct processing stages familiar to the information retrieval field.

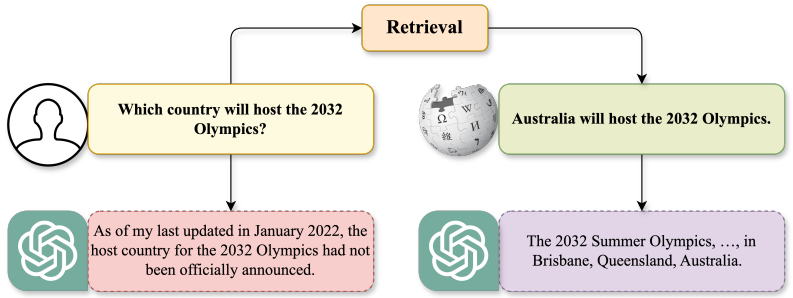


Fig. 1. An example of the benefits of RAG: ChatGPT resolves questions that fall outside its training data and produces correct answers.

To provide a clear structure, this article is organized as follows: The overall RAG workflow is described in Section 2, where the methods are divided into the pre-retrieval, retrieval, post-retrieval, and generation phases. The core techniques within each phase are examined in Sections 3, 4, 5, and 6. The evaluation methods for RAG are presented in Section 7, while challenges and future research directions are discussed in Section 8. A conclusion is provided in Section 9.

2 RAG Framework

Hallucination is largely attributed to LLMs' inability to access up-to-date information. This limitation stems from the models' reliance on their training datasets. RAG addresses this issue by supplementing the LLM's internal knowledge with current information from external sources via a retrieval model, thereby enabling the generation of more accurate responses. RAG also offers a more cost-effective alternative to the extensive training and fine-tuning processes typically required for LLMs. It supports the dynamic incorporation of new information via traditional retrieval methods or pre-trained LMs, without directly integrating this data into the LLM. This feature makes RAG both flexible and scalable, facilitating its application across different LLMs and use cases. The information retrieved through RAG comes from real-world, human-authored data, which not only simplifies the generation process but also increases the reliability of the generated responses.

Research by Khandelwal et al. [82] demonstrates that accessing relevant information from the training dataset itself can significantly improve LLM performance, highlighting the effectiveness of RAG. Over time, RAG has evolved from a method for providing supplementary information to one that supports multiple interactions between the retrieval and generation components. These approaches conduct several rounds of retrieval to improve the relevance and accuracy of the retrieved information and, in turn, iteratively enhance the quality of the generated output. Toolkits such as LangChain[1] and LlamaIndex[2] have modularized the RAG approach, increasing its adaptability and expanding its range of applications. Although these toolkits employ diverse methodologies to tackle different aspects of RAG, ranging from multiple search iterations to iterative generation, they maintain adherence to the basic workflow. This consistency is crucial for understanding how they operate and for identifying opportunities for further development.

2.1 Basic RAG Workflow

Figure 2 represents the unified RAG core concepts with a basic workflow. The workflow of RAG begins with the creation of an index comprising external sources. This index serves as the basis for

[1] https://www.langchain.com
[2] https://www.llamaindex.ai

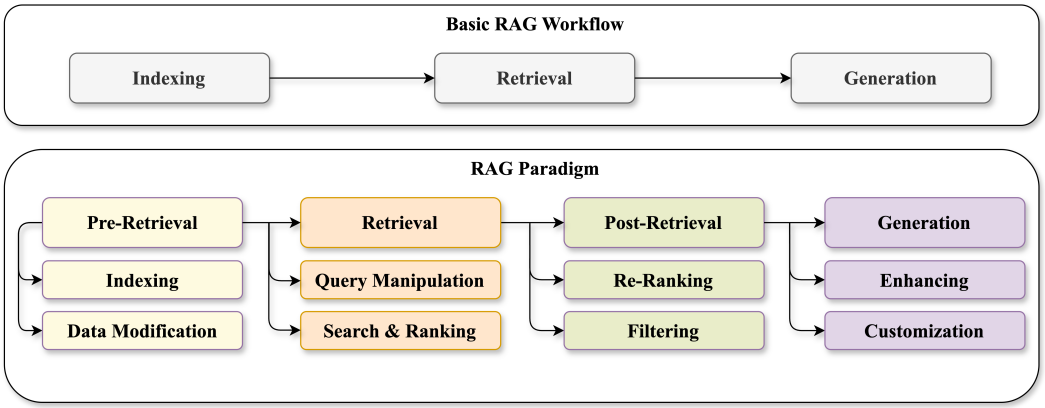


Fig. 2. The unified RAG core concepts with basic workflow.

retrieving relevant information through a retriever model based on a specific query. The final step involves a generator model, which combines the retrieved information with the query to produce the desired output.

2.1.1 Indexing. Efficient retrieval begins with comprehensive indexing, where data preparation is key. This stage involves text normalization processes such as tokenization, stemming, and the removal of stop words to enhance the text's suitability for indexing [113]. Text segments are then organized into sentences or paragraphs to facilitate more focused searches, allowing for the pinpointing of segments containing pertinent keywords. The integration of deep learning has revolutionized indexing through the use of pre-trained LMs for generating semantic vector representations of texts. These vectors are stored, enabling rapid and precise retrieval from extensive data collections, significantly enhancing retrieval efficiency.

2.1.2 Retrieval. While traditional retrieval methods, such as the BM25 algorithm [50, 60], focus on term frequency and presence for document ranking, they often overlook the semantic information of queries [123]. Current strategies leverage pre-trained LMs like BERT [32], which capture the semantic essence of queries more effectively. These models improve search accuracy by considering semantically similar words and the structure of phrases, thereby refining document ranking through the detection of semantic similarities. This is typically achieved by measuring vector distances between documents and queries, combining traditional keyword-based models with semantic understanding capabilities to yield search results that are both relevant and aligned with user intent.

2.1.3 Generation. The generation phase is tasked with producing text that is both relevant to the query and reflective of the information found in the retrieved documents. The usual method involves concatenating the query with the retrieved information, which is then fed into an LLM for text generation [93]. Although ensuring the generated text's alignment and accuracy with the retrieved content presents challenges, it is essential to strike a balance between adhering closely to the source material and infusing the output with creativity. The generated text should accurately convey the information from the retrieved documents and align with the query's intent, while also offering the flexibility to introduce new insights or perspectives not explicitly contained within the retrieved data.

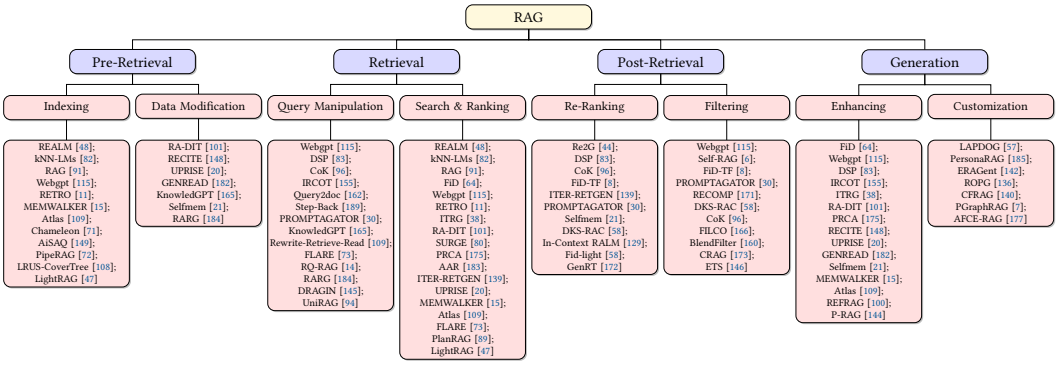


Fig. 3. Taxonomy tree of RAG's core techniques

2.2 RAG Paradigm

The RAG paradigm organizes research within the domain, offering a straightforward yet robust framework to enhance LLM performance. Central to RAG is its search mechanism, crucial for generating high-quality outcomes. Therefore, this paradigm is structured into four main phases from a retrieval perspective: pre-retrieval, retrieval, post-retrieval, and generation. Both single-hop and multi-hop retrieval approaches, encompassing iterative retrieve-generate cycles, follow this four-phase structure. Figure 3 is the taxonomy tree of RAG's core techniques.

2.2.1 Pre-Retrieval. The pre-retrieval phase of retrieval-augmented generation lays the foundation for successful data preparation, ensuring efficient information retrieval. This phase includes essential tasks to prepare for effective data access.

Indexing. The process starts with indexing, which establishes an organized system to enable fast and accurate retrieval of information. The specificity of indexing depends on the task and data type. For example, sentence-level indexing is beneficial for question-answering systems to precisely locate answers, while document-level indexing is more appropriate for summarizing documents to understand their main concepts and ideas.

Data Modification. Data modification is also critical in enhancing retrieval efficiency. This step includes preprocessing techniques like removing irrelevant or redundant information to improve the quality of results and enriching the data with additional information, such as metadata to boost the relevance and diversity of the retrieved content [9].

2.2.2 Retrieval.

Query Manipulation. Query manipulation is performed to adjust user queries for a better match with the indexed data. This involves query reformulation [68, 180], which rewrites the query to align more closely with the user's intention; query expansion [56], which extends the query to capture more relevant results through synonyms or related terms; and query normalization, which resolves differences in spelling or terminology for consistent query matching.

Search and Ranking. The retrieval stage is a combination of search and ranking. It focuses on selecting and prioritizing documents from a dataset to enhance the quality of the generation model's outputs. This stage employs search algorithms to navigate through the indexed data,

finding documents that match a user's query. After identifying relevant documents, the process of initially ranking these documents starts to sort them according to their relevance to the query.

2.2.3 Post-Retrieval. The post-retrieval phase serves to refine the initially retrieved documents to improve the quality of text generation. This phase consists of re-ranking and filtering, each aimed at optimizing the document selection for the final generation task.

Re-Ranking. In the re-ranking step, the documents previously retrieved are reassessed, scored, and reorganized. The objective is to more accurately highlight the documents most relevant to the query and diminish the importance of the less relevant ones. This step involves incorporating additional metrics and external knowledge sources to enhance precision. In this context, pre-trained models with superior accuracy but lower efficiency can be effectively employed due to the limited set of candidate documents available [59].

Filtering. Filtering aims at removing documents that fail to meet specified quality or relevance standards [61, 84]. This can be done through several approaches, such as establishing a minimum relevance score threshold to exclude documents below a certain relevance level. Furthermore, the use of feedback from users or prior relevance evaluations assists in adjusting the filtering process, guaranteeing that only the most relevant documents are retained for text generation.

2.2.4 Generation. The generation stage is a crucial component of the RAG process, responsible for leveraging retrieved information to enhance the quality of the generated response. This stage encompasses several sub-steps aimed at producing content that is readable, engaging, and informative.

Enhancing. At the heart of the generation phase is the enhancement step, where the objective is to merge the retrieved information with the user's query to create a coherent and relevant response. This includes the process of elaboration, adding extra details to the retrieved content to enrich it. Efforts are focused on improving the output's quality by increasing its clarity, coherence, and stylistic appeal through methods such as rephrasing and restructuring. Information from various sources is combined to offer a comprehensive perspective, and verification is conducted to ensure the accuracy and relevance of the content.

Customization. Customization is user-centric. It encompasses tailoring content in two primary ways. First, it aligns the generated output with relevant information retrieved in earlier stages, ensuring consistency and accuracy by incorporating key knowledge. Second, it adapts the content to suit user-specific factors such as intended audience, situational context, and personal preferences, shaping the response to be both contextually relevant and user-centric. This dual focus on integrating relevant knowledge and adjusting to diverse contextual demands forms the basis of effective customization in RAG.

3 Pre-Retrieval

3.1 Indexing

An index is a data structure that organizes information for efficient access. In traditional information retrieval, the inverted index is a common structure, which maps words to the documents that contain them, allowing for fast lookups. This approach, however, struggles to find documents that are semantically related to a query but use different terms. To solve this problem, RAG systems typically use indexing based on dense vector embeddings. The main challenge is that creating and searching these vectors for very large document collections can be slow and require a lot of computational power. A core strategy to manage this is **Approximate Nearest Neighbor Search**

(**ANNS**) [4], which accepts a small reduction in retrieval accuracy for a large improvement in speed. The main ANNS indexing strategies can be grouped into graph-based, quantization-based, and hashing-based methods, each offering different balances between accuracy, memory use [69], and query speed [31].

Graph. Graph-based indexing is a foundational technique in RAG for accelerating vector retrieval. By structuring vector indexes as graphs, retrieval operations can be constrained to a local subgraph, significantly reducing the computational cost of distance calculations and thus enhancing search speed. This principle underpins several prominent methods. For instance, **k-nearest neighbor language models** (**kNN-LMs**) augment pre-trained models with a datastore of text collections, enabling dynamic retrieval of relevant examples without further training [82]. To implement this efficiently, libraries like FAISS [78] are widely adopted [82, 83, 91], often incorporating approximate nearest neighbor algorithms such as **Hierarchical Navigable Small World** (**HNSW**) [111] to further optimize retrieval performance [91].

Beyond serving as general-purpose search accelerators, recent research has begun to tailor graph structures to solve specific challenges inherent to LLMs. Instead of using generic graph structures, LightRAG [47] uses an LLM to extract entities and their relationships to build a knowledge graph. This approach moves away from simple data representations and helps capture complex connections between different pieces of information. To combat out-of-date knowledge, WebGPT [115] employs a dynamic indexing strategy that leverages the Bing search API to build its knowledge base from up-to-date user search histories. To address the limited context window of LLMs, MEMWALKER [15] conceptualizes the index not as a flat vector space but as a hierarchical memory tree that reflects the semantic structure of long documents. Moreover, work continues on optimizing the core graph algorithms. Methods like LRUS-CoverTree [108] target the high cost of index construction in **k-Maximum Inner-Product Search** (**k-MIPS**) scenarios, which are common for LLM embeddings, achieving comparable search performance with significantly faster build times. These advancements demonstrate an evolution of graph-based indexing from a pure efficiency tool to a versatile framework for addressing core LLM limitations.

Product Quantization. In contrast to graph methods that limit the number of comparisons, **Product Quantization** (**PQ**) [69] accelerates vector search by reducing the cost of each individual comparison. By segmenting vectors and representing them with compact, learned codes, PQ achieves significant data compression, which in turn lowers memory usage and speeds up distance calculations. For RAG systems, PQ's contribution is not better accuracy but radical resource efficiency, a prerequisite for massive-scale deployment. Recent work highlights this focus on systems engineering. AiSAQ [149], for example, uses PQ to offload a billion-vector index from DRAM to cheaper storage, reducing its footprint to megabytes. Likewise, PipeRAG [72] and Chameleon [71] embed PQ within distributed pipelines to maximize end-to-end throughput. Such examples demonstrate that practical RAG at scale depends equally on systems engineering and algorithmic quality, paving the way for economically feasible, large-corpus deployments beyond resource-rich institutions.

Locality-Sensitive Hashing. The core idea of **Locality-Sensitive Hashing** (**LSH**) is to place similar vectors into the same hash bucket with high probability [3]. LSH uses hash functions that map similar vectors to the same or nearby hash values, making it easier to find approximate nearest neighbors. In LSH, when a query vector is hashed, the system quickly retrieves candidate vectors that share the same hash value. This method reduces the dimensionality of the problem and can be implemented efficiently, but it may introduce some inaccuracies due to the hashing process itself

[66]. While LSH is rarely used in RAG systems compared to graph-based and PQ methods, it still offers a useful approach in scenarios where speed is prioritized over the slight loss in accuracy.

3.2 Data Modification

The effectiveness of RAG depends heavily on the quality and relevance of the data used for augmentation. The data can originate from two fundamental sources: the internal knowledge stored within the language model or external knowledge bases. Choosing between these sources involves a tradeoff. Using internal knowledge makes the system self-contained but limits it to what the model has already learned. Using external data provides access to current and specific information but adds the complexity of managing a retrieval system.

Internal Data Augmentation. The first group of methods leverages the knowledge the large language model has already acquired during its training. Instead of retrieving from an outside source, these techniques prompt the model to generate its own context. This approach treats the model not only as a final response generator but also as a source of information. For instance, methods like RECITE [148], KnowledGPT [165], and GENREAD [182] are all based on a similar principle. They make the model's implicit parametric knowledge explicit by generating relevant text that is then used as context to produce the final answer. This strategy is useful because it does not depend on an external database, which can simplify the system architecture. However, the main weakness is that it is restricted to the knowledge frozen at the time of the model's training. It cannot provide information that is more recent than its training data, and it might even generate incorrect context if the model's internal knowledge is flawed. A more dynamic internal approach is seen in Selfmem [21], which creates an iterative process. The model uses its own previous outputs as a form of memory for future generation tasks. This demonstrates a shift from a single context generation step to a self-improving loop, where the model refines its knowledge based on its own reasoning. Across these methods, a clear pattern is the increasing use of the LLM's own generative abilities to create or refine its context, though all are ultimately limited by the scope of their pre-trained knowledge.

External Data Enrichment. The second group of methods enriches the generation process by drawing information from external sources. This strategy directly addresses the limitations of using only internal knowledge, such as outdated information or factual inaccuracies. By grounding the model in external, verifiable documents, these techniques aim at improving the accuracy and timeliness of the generated responses. The approaches to using external data vary. Some methods focus on improving how the model interacts with external information. RA-DIT [101] fine-tunes the model with prompts that are already augmented with documents from large corpora like Wikipedia. This trains the model to become better at using retrieved information. Similarly, UPRISE [20] improves generalization by retrieving effective prompts from a collection of different task datasets, which helps the model perform better on new, unseen tasks without specific training. These methods essentially teach the model to be a more effective user of external context. Other methods focus on the direct application of retrieval for specific tasks. RARG [184], for instance, retrieves evidence from scientific databases to generate responses that counter misinformation. This represents a classic RAG pattern where the goal is to ground the output in a specific, authoritative knowledge source. The primary strength of using external data is the ability to provide factual, up-to-date, and domain-specific information, which is a direct way to reduce model hallucination. The main limitation, however, is the system's dependency on the retriever. If the retrieval step fails to find relevant documents or pulls in noisy, irrelevant information, the quality of the final generated answer will likely decrease.

4 Retrieval

4.1 Query Manipulation

Query manipulation aims at improving retrieval by modifying the user's original query. The main goal is to reduce the gap between how a user expresses a need and how relevant information is described in the documents. This is important in RAG as the quality of the retrieved documents directly affects the quality of the final generated text. Query manipulation techniques can be grouped into three main strategies. One strategy is query expansion, which adds new, related terms to the query to broaden its scope. Another is query reformulation, which rephrases the original query to improve its clarity and precision. A third strategy is prompt-based rewriting, which uses the instructions given to LLMs to guide how the query is understood and processed.

Query Expansion. Query expansion makes a query broader by adding related terms to it. This method helps find more relevant documents, which is useful when the original query terms do not exactly match the terms in the documents. One way to do this is to use the LLM itself to generate new terms. Query2doc [162] instructs an LLM to generate a full pseudo-document that seems like a good answer to the query. This generated text is then used to enrich the original query, helping both sparse and dense retrieval systems find documents with similar content. Another way to expand a query is by using external information. Some methods retrieve an initial set of documents and then use the information from those documents to expand the query for a second, more thorough retrieval. For instance, FiD [64] retrieves many passages to give the model more evidence to work with. The RARG [184] framework also uses supporting documents as evidence to create a more informed query, which is particularly helpful for complex tasks like countering misinformation. Other methods, such as KnowledGPT [165], connect to external knowledge bases to add structured information to the query. These approaches show two different philosophies for expansion: one relies on the LLM's generative ability to produce relevant content, while the other grounds the expansion in existing data, either from an initial retrieval pass or a structured knowledge source.

Query Reformulation. In contrast to adding new terms, query reformulation modifies the original query to make it clearer and more precise. This helps the retrieval system better understand the user's intent. Some reformulation methods perform a one-time analysis of the query before retrieval. For example, RQ-RAG [14] is designed for complex queries and works by rewriting, breaking down, and clarifying the user's input to create a more specific query. This approach helps ensure that the reformulated query more directly points to the needed information. Other methods use an iterative approach to reformulation. The Rewrite-Retrieve-Read framework [109] first refines the query to improve the initial retrieval and then uses the retrieved information to generate a better response. A similar iterative process is used by FLARE [73], which refines the query as part of a cycle between retrieving information and generating text. This method uses low-confidence tokens in generation as a signal to reformulate a query and retrieve more information. The comparison shows a difference in strategy: the one-time method aims at perfecting the query before starting, which can be faster, while the iterative method refines the query based on what is found, which can be more adaptive but may require more steps and be less efficient.

Prompt-based Rewriting. Prompt-based rewriting uses detailed instructions, or prompts, to change how an LLM interprets a query. Instead of directly altering the query's words, this method places the query inside a larger context that guides the model's reasoning. This is different from expansion and reformulation because it focuses on controlling the model's thought process rather than just creating a better search term. One use of this method is to make the model reason at a higher level of abstraction before retrieval. For instance, the Step-Back [189] prompting technique instructs

the model to first generate a more general question or concept related to the user's specific query. Another use of prompts is to manage the interaction with different knowledge sources. The CoK [96] method uses prompts to guide the model on which knowledge source to use and how to integrate the information, effectively rewriting the query's context based on the available data. Prompts can also be used to generate better queries for specific tasks. Promptagator [30] uses prompts in few-shot scenarios to guide the model in creating new queries that fit the task, adapting the retrieval process without needing a large training dataset. While individual strategies like expansion and reformulation are effective, they are often applied separately. Recent research has started to address this by creating frameworks that can choose the best manipulation strategy. UniRAG [94] proposes a unified framework that learns to adaptively select the most suitable strategy for each query by using feedback from the retrieval and generation results.

4.2 Search and Ranking

The search and ranking process within RAG is important for improving the relevance and accuracy of generated outputs. A primary challenge is to ensure that the retriever fetches documents that are not just semantically similar to the query, but are also useful for the generator. One group of methods addresses this by using the generator itself to provide supervision for the retriever. Atlas [65] focuses on optimizing the retriever's ability to select contextually relevant documents, especially in new domains with limited data, by employing few-shot learning techniques such as Attention Distillation and Perplexity Distillation. In contrast, AAR [183] adapts retrieval preferences to better align with the requirements of LLMs, enhancing retrieval generalization across tasks by training a smaller source model.

A second group of methods moves beyond a single retrieval step to create a dynamic and iterative interaction between retrieval and generation. This approach, sometimes called Loop RAG, is particularly helpful for complex queries that require multiple pieces of information [43]. For instance, IRCOT [155] interleaves retrieval with chain-of-thought reasoning, using each reasoning step to guide a new search for evidence. Other methods use different triggers for retrieval. FLARE [73] prompts retrieval only when the generator's confidence in its next word is low, allowing the model to seek help when it is uncertain. Similarly, Self-RAG [6] learns to generate special tokens that decide when to retrieve documents and whether the retrieved information is relevant. The framework of ITER-RETGEN [139] formalizes this loop by alternating between using generated text to refine retrieval and using retrieved information to improve the next generation step.

Other methods improve retrieval by using structured knowledge instead of simple text chunks. To achieve this, some methods use knowledge graphs to show the relationships between different pieces of information. SURGE [80] uses a subgraph retriever to navigate these graphs, finding relevant facts based on the explicit connections between entities. A more recent approach, LightRAG [47], also builds a graph-based index but uses a dual-level retrieval system. This system retrieves both low-level information about specific entities and high-level information about broader topics, which helps provide a more complete context for the generator. These methods show how specialized data structures can improve the quality of retrieved information compared to working with unstructured text alone.

As more documents are retrieved, another challenge is managing the large volume of text provided to the generator. Many approaches focus on post-retrieval processing to solve this problem. One common technique is re-ranking documents to place the most important content at the beginning and end of the context window. This helps avoid the "lost in the middle" effect, where models tend to ignore information in the middle of a long input [103]. Other methods use compression to remove irrelevant or redundant information from the retrieved text before it is passed to the generator [70, 171]. For tasks where the generator is a black-box model and cannot be modified,

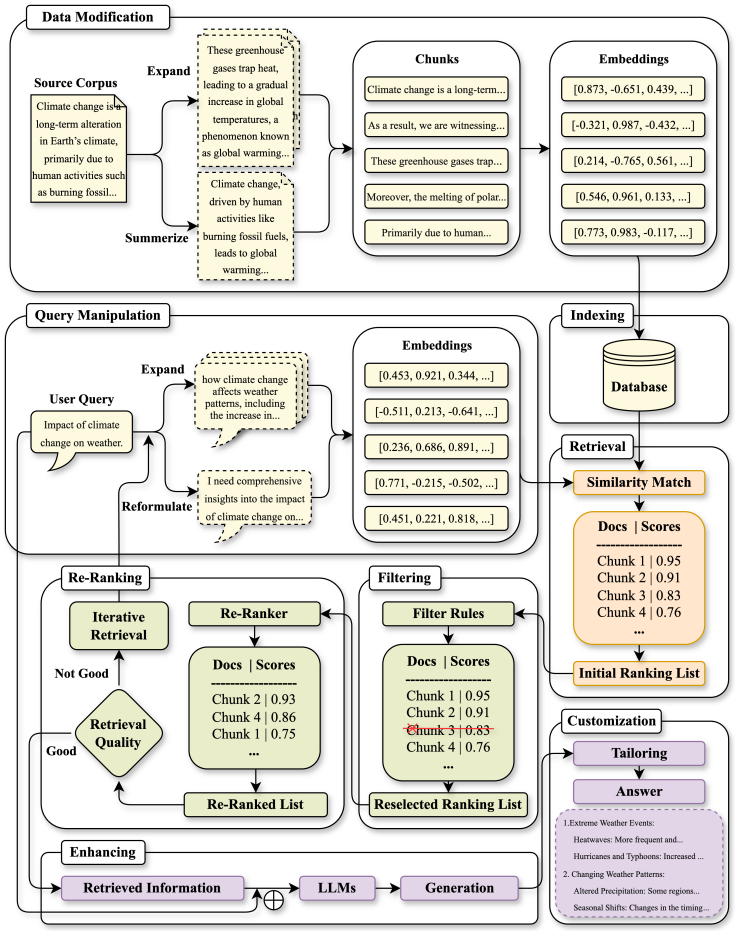


Fig. 4. An example of a common RAG framework with an iterative retrieval strategy.

PRCA [175] uses a reward-driven adapter to learn how to compress and summarize retrieved documents into a format that is more useful for the generator. A different strategy is presented by MEMWALKER [15], which organizes information into a hierarchical memory tree. It then uses an internal search and ranking mechanism to navigate this structure, helping the model to efficiently locate key information in long contexts.

4.3 Retrieval Strategy

The strategies for retrieval in RAG are important for tailoring the process to fit different application needs. These strategies have moved from simple, single-step searches to more complex, dynamic approaches that determine when, what, and how to retrieve information. Instead of focusing on developing new retrieval algorithms, most RAG research concentrates on how best to use existing retrieval techniques. While early RAG models performed retrieval only once, current methods often use multi-hop searches, where information is gathered through several steps. This allows the system to build context, handle complex questions, and improve the quality of the evidence provided to the generator. Figure 4 shows a common RAG setup using an iterative retrieval strategy, while Table 1 outlines the five main retrieval strategies discussed below.

Table 1. Comparison of Five RAG Retrieval Strategies

Strategy	Core Mechanism	Primary Use Case	Complexity	Cost	Key Strength
Basic	Retrieve-then-read	Simple, factual question-answering.	Low	Low	Simplicity and cost-effectiveness.
Iterative	Sequentially decomposes a query into simpler retrieval steps.	Multi-hop reasoning and finding intermediate facts.	Medium	Medium to High	Models explicit, step-by-step reasoning paths.
Recursive	Indexes knowledge in a tree/graph for multi-level retrieval.	Document summarization and holistic understanding.	High	Medium	Provides multi-scale context (from broad to specific).
Conditional	Uses rules or an evaluator to decide when to retrieve.	Variable-complexity queries; optimizing for efficiency.	Medium	Low to Medium	Avoids unnecessary retrieval, reducing cost and latency.
Adaptive	LLM dynamically decides when and what to retrieve during generation.	Complex, open-ended tasks needing on-demand information.	High	High	Highly flexible; addresses knowledge gaps on-the-fly.

Basic Retrieval Strategy. The most direct method is the basic retrieval strategy, which follows a linear workflow. This process moves sequentially from an initial query to retrieval, followed by the generation phase, without any loops or complex adjustments. This single-pass approach is valued for its simplicity and speed. For example, frameworks like Atlas [65] and REPLUG [141] show this direct approach. REPLUG treats the language model as a fixed component and enhances it with information from a single retrieval step, without iterative loops or complex interactions. The main limitation of this basic strategy is its performance on complex questions. When an answer requires putting together information from multiple sources or different parts of a long document, a single retrieval pass may not collect all the necessary context, leading to incomplete or incorrect generated answers.

Iterative Retrieval Strategy. The iterative retrieval strategy uses a loop between retrieval and generation to handle more complex questions (Algorithm 1). In this approach, the system retrieves information multiple times, and each retrieval step can be guided by the output of the previous generation step. This creates a feedback mechanism that allows the system to refine its search and build a more complete context over several iterations. This strategy is effective for tasks that require multi-step reasoning, as it can gather evidence piece by piece. For example, IRCOT [155] combines retrieval with chain-of-thought reasoning by performing a new retrieval for each step in the reasoning chain. This grounds each intermediate thought with external evidence. In contrast, ITER-RETGEN [139] uses the entire generated response from one step to create a new, better query for the next retrieval step, refining the overall context. Other methods like RQ-RAG [14] and PlanRAG [89] also use iterative refinement, where the system continuously improves its query or plan based on the information it gathers, making the retrieval process more targeted with each iteration.

Recursive Retrieval Strategy. Unlike the linear process of iterative retrieval, recursive retrieval (Algorithm 2) is designed for information that has a hierarchy or layered structure, like a document with sections and subsections, or a knowledge graph. This strategy addresses complex queries by breaking them down and navigating from general information to specific details. The adjustment of the query at each hierarchical layer is not always achieved by rewriting the query, but by narrowing the search space. For example, in systems that organize documents into a tree of summaries like RAPTOR [138], an initial query might match a high-level summary. The next retrieval step is then focused only on the more detailed text chunks that make up that summary. The query remains the same, but it is applied to a much smaller and more relevant set of data. Similarly, in graph-based

ALGORITHM 1: Iterative Retrieval Strategy in RAG

Require: Query q, Documents D, Maximum Iterations N, Retriever R, Generator G, Pre-retrieval Function F_{pre}, Query Manipulation Function F_{qm}, Post-retrieval Function F_{post}
Ensure: Final Output y_{final}

Offline Stage:
Pre-retrieval Phase
$D_{idx} \leftarrow F_{pre}(D)$ *// Indexing and Data Modification*
Online Stage:
Initialize $i \leftarrow 1$ *// Start iteration counter*
while $i \leq N$ **do**
  Retrieval Phase
  $q' \leftarrow F_{qm}(q)$ *// Query Manipulation before search*
  $D_i \leftarrow R(q', D_{idx})$ *// Search and initial ranking of documents*
  Post-retrieval Phase
  $D_i' \leftarrow F_{post}(q', D_i)$ *// Re-ranking and filtering to refine documents*
  Generation Phase
  $y_i \leftarrow G(q', D_i')$ *// Generate output based on refined documents*
  if stopping condition met based on y_i **then**
    BREAK *// Stop iterations if output is satisfactory*
  end if
  Update $q \leftarrow$ UpdateQuery(q', y_i) *// Refine query for the next iteration*
  $i \leftarrow i + 1$ *// Increment iteration counter*
end while
Final Synthesis
$y_{final} \leftarrow$ SynthesizeResults$(\{y_1, y_2, \ldots, y_i\})$ *// Merge results*
return y_{final}

methods such as SURGE [80], an initial retrieval may identify key entities in a knowledge graph. Subsequent retrieval steps then explore the nodes directly connected to those entities, with the query becoming an exploration of the graph's local structure. This strategy is powerful for "zooming in" on the correct information within structured knowledge bases. Its main requirement is that the knowledge must be pre-organized into a hierarchical format, which is not always practical.

Conditional Retrieval Strategy. Conditional strategies (Algorithm 3) introduce pre-defined rules or decision points that determine if and when retrieval should occur. This approach improves efficiency by avoiding unnecessary retrieval when the language model's internal knowledge is sufficient to answer a query. The decision to retrieve is often based on a specific trigger. For instance, PRCA [175] uses a reward-driven mechanism to decide whether additional context would be helpful. In contrast, CRAG [173] employs a retrieval evaluator that measures the confidence of initial results and triggers different actions, such as searching for more information if confidence is low. RARG [184] makes retrieval conditional on finding specific types of evidence, which is useful for generating factual and polite responses. The main advantage of this strategy is reduced computational cost and latency. However, its effectiveness depends entirely on the accuracy of the decision-making module that triggers the retrieval.

Adaptive Retrieval Strategy. Adaptive retrieval (Algorithm 4) gives the language model more control to dynamically adjust the retrieval process based on the ongoing context of a query. Unlike iterative or conditional methods that follow more structured loops or fixed rules, adaptive strategies can alter the retrieval method itself during generation. For instance, the model might decide to broaden a query, switch from a vector search to a keyword search, or actively seek new information

ALGORITHM 2: Recursive Retrieval Strategy in RAG

Require: Query q, Documents D, Maximum Depth L, Retriever R, Generator G, Current Search Space D_c, Query Manipulation Function F_{qm}, Post-retrieval Function F_{post}, Hierarchy Building Function F_{build}
Ensure: Final Output y_{final}

Offline Stage:
Pre-retrieval Phase
$D_{hier} \leftarrow F_{build}(D)$ *// Build a hierarchical or structured version of the corpus*
Online Stage:
Initialize $l \leftarrow 1$ *// Start depth counter*
Initialize $D_c \leftarrow \text{TopLevel}(D_{hier})$ *// Start current search space at the highest level of the hierarchy*
while $l \leq L$ **do**
  Retrieval Phase
  $q' \leftarrow F_{qm}(q)$ *// Query Manipulation for the current level*
  $D_l, D'_c \leftarrow R(q', D_c)$ *// Retrieve documents and determine the next search space for the next level*
  Post-retrieval Phase
  $D'_l \leftarrow F_{post}(q', D_l)$ *// Re-rank and filter documents for the current level*
  Generation Phase
  $y_l \leftarrow G(q', D'_l)$ *// Generate output based on the current level's documents*
  Update for Next Level
  $D_c \leftarrow D'_c$ *// Narrow the search space for the next iteration*
  Update $q \leftarrow \text{UpdateQuery}(q', y_l)$ *// Optionally refine the query for the next level*
  $l \leftarrow l + 1$ *// Increment depth counter*
end while
Final Synthesis
$y_{final} \leftarrow \text{SynthesizeResults}(\{y_1, y_2, \ldots, y_{l-1}\})$ *// Merge results*
return y_{final}

when it detects low confidence in its generated text. FLARE [73] is a representative example, as it actively decides when to retrieve more information by monitoring the probability of the tokens it is generating. Self-RAG [6] gives the model the ability to reflect on its own output and decide whether to retrieve information or to critique its generated text using special tokens. CoK [96], on the other hand, implements a dynamic mechanism that adjusts retrieval strategies based on the evolving needs of the task. The retrieval process in CoK is not static but adapts according to the specific scenario and the nature of the information being accessed, making it highly effective for context-sensitive applications. These adaptive methods provide high flexibility and can lead to more precise retrieval, but they also increase the complexity of the overall system.

In summary, retrieval strategies represent a range of methods for different needs and complexities. The Basic strategy offers speed and simplicity. Iterative and Recursive strategies provide greater depth and are better for complex, multi-hop questions, but they introduce higher latency and computational costs. Conditional and Adaptive strategies offer a more intelligent approach by adding a layer of decision-making to the retrieval process. They aim at optimizing the balance between performance and efficiency, making retrieval more targeted and effective. The choice of strategy depends on the specific requirements of the task, such as the need for speed, depth of information, or the ability to handle dynamic and complex queries.

5 Post-Retrieval

5.1 Re-Ranking

The initial retrieval stage is designed for speed and typically returns a broad set of documents that may be relevant. A re-ranking stage is therefore used to refine this initial list, ordering the

ALGORITHM 3: Conditional Retrieval Strategy in RAG

Require: Query q, Documents D, Maximum Iterations N, Retriever R, Generator G, Pre-retrieval Function F_{pre}, Query Manipulation Function F_{qm}, Post-retrieval Function F_{post}, Condition Evaluation Function F_{cond}
Ensure: Final Output y_{final}

Offline Stage:
Pre-retrieval Phase
$D_{idx} \leftarrow F_{pre}(D)$ *// Indexing and Data Modification*
Online Stage:
Initialize $i \leftarrow 1$ *// Start iteration counter*
while $i \leq N$ **do**
  Retrieval Phase
  $q' \leftarrow F_{qm}(q)$ *// Query Manipulation before search*
  $D_i \leftarrow R(q', D_{idx})$ *// Search and initial ranking of documents*
  Post-retrieval Phase
  $D_i' \leftarrow F_{post}(q', D_i)$ *// Re-rank and filter documents*
  Generation Phase
  $y_i \leftarrow G(q', D_i')$ *// Generate output based on refined documents*
  Conditional Branching
  if $F_{cond}(y_i, D_i')$ is Condition A **then**
    Apply Strategy A *// e.g., refine the query based on feedback*
  else if $F_{cond}(y_i, D_i')$ is Condition B **then**
    Apply Strategy B *// e.g., expand the scope or adjust parameters*
  else if $F_{cond}(y_i, D_i')$ is Condition C **then**
    Apply Strategy C *// e.g., modify retrieval strategy or output processing*
  else
    Continue *// If no conditions are met, proceed without adjustments*
  end if
  Check Termination Condition
  if $F_{cond}(y_i, D_i')$ meets stopping criteria **then**
    BREAK *// Exit the loop if the stopping condition is met*
  end if
  $i \leftarrow i + 1$ *// Increment iteration counter*
end while
Final Synthesis
$y_{final} \leftarrow \text{SynthesizeResults}(\{y_1, y_2, \ldots, y_i\})$ *// Merge results*
return y_{final}

documents more precisely according to their relevance to the query. This step is important because the quality of the documents passed to the generator directly influences the accuracy and coherence of the final answer. Research in this area has explored several strategies for re-ranking, which can be broadly categorized through three main approaches: unsupervised methods that use the inherent knowledge of LLMs, supervised methods that train models on specific ranking data, and data augmentation techniques that create training data to improve supervised models [192].

Unsupervised Re-ranking. Unsupervised re-rankers are useful when labeled ranking data is not available. These methods use the inherent knowledge of LLMs to estimate document relevance without needing to be trained on a specific ranking task. There are three common approaches. In the pointwise approach, the model scores each document independently. For example, In-Context RALM [129] uses a standard language model to re-rank documents retrieved by a BM25 retriever in a zero-shot manner. It determines the best document by selecting the one that makes the generation of

ALGORITHM 4: Adaptive Retrieval Strategy in RAG

Require: Query q, Documents D, Maximum Iterations N, Retriever R, Generator G, Pre-retrieval Function F_{pre}, Query Manipulation Function F_{qm}, Post-retrieval Function F_{post}, Adaptive Adjustment Function F_{adapt}, Feedback Function $F_{feedback}$

Ensure: Final Output y_{final}

Offline Stage:
Pre-retrieval Phase
$D_{idx} \leftarrow F_{pre}(D)$ *// Indexing and Data Modification*
Online Stage:
Initialize $i \leftarrow 1$ *// Start iteration counter*
$q, Context \leftarrow q, \emptyset$ *// Initialize query and context*
while $i \leq N$ **do**
  Retrieval Phase
  $q', Context \leftarrow F_{qm}(q, Context)$ *// Query Manipulation and Context Setup before search*
  $D_i \leftarrow R(q', D_{idx}, Context)$ *// Retrieve documents based on the current query and context*
  Post-retrieval Phase
  $D_i' \leftarrow F_{post}(q', D_i, Context)$ *// Re-ranking and filtering based on adaptive criteria*
  Generation Phase
  $y_i \leftarrow G(q', D_i', Context)$ *// Generate output based on refined documents*
  Adaptive Adjustment
  if $F_{feedback}(y_i, D_i')$ is negative **then**
    $q, Context \leftarrow F_{adapt}(q', Context, y_i, D_i')$ *// Dynamically adjust query and context*
  end if
  Feedback Integration
  if $F_{feedback}(y_i)$ is positive **then**
    BREAK *// Stop iterations if output is satisfactory*
  end if
  $i \leftarrow i + 1$ *// Increment iteration counter*
end while
Final Synthesis
$y_{final} \leftarrow \text{SynthesizeResults}(\{y_1, y_2, \ldots, y_i\})$ *// Merge results*
return y_{final}

the text most likely. While simple and efficient, pointwise methods do not consider the relationships between different documents. The listwise approach addresses this by providing the model with a list of documents and asking it to generate an ordered list based on relevance [147]. This allows the model to make comparisons across all provided documents. However, this method can be sensitive to the initial order of documents and is limited by the model's maximum input length. The pairwise approach offers a balance by asking the model to compare two documents at a time and decide which is more relevant to the query [126]. By performing a series of these comparisons, a final ranked list can be produced. This method captures relative relevance better than pointwise methods, but can be computationally intensive as the number of pairs grows.

Supervised Re-ranking. Supervised re-rankers are fine-tuned on datasets with explicit relevance labels, which generally results in higher accuracy for specific domains. Early approaches used models like BERT as cross-encoders to process a query and a document together to produce a relevance score [44]. While accurate, this method is computationally expensive because each document must be evaluated individually with the query. Other methods use encoder-decoder models like T5 to treat ranking as a text generation task, where the model generates a token like "true" or "false" to indicate relevance [117]. Building on this idea, FiD-Light [53] uses a listwise

auto-regressive mechanism to re-rank passages by generating an ordered list of pointers to the source documents. Similarly, GenRT [172] combines an encoder to capture features across the entire list of documents with a decoder that sequentially reorders them. More recent prompt-based techniques, like RankLLaMA [110], calculate relevance by focusing on the representation of the last generated token. A notable pattern in this area is using a powerful re-ranker to improve other parts of the RAG pipeline through knowledge distillation. In the ITER-RETGEN [139] framework, a strong re-ranker that can access the generator's output is used to teach the initial retriever. This process trains the retriever to better predict which documents will be most useful for the generator, creating a more cohesive and effective system.

Data Augmentation for Re-ranking. The effectiveness of supervised re-rankers depends heavily on the quality and relevance of their training data. However, standard information retrieval datasets often do not fully meet the requirements of generative tasks, which has led to the development of data augmentation techniques. These techniques serve as a practical middle ground that uses LLMs to generate synthetic training examples, such as pseudo-relevance labels [178], which can then be used to train a more specialized or efficient re-ranking model. One strategy is to generate entirely new training examples, like PROMPTAGATOR [30], which creates synthetic queries for existing documents. This process builds a larger set of query-document pairs, helping the re-ranker learn to identify relevant documents more effectively by training on a greater variety of examples. A different strategy does not create new pairs but instead enriches existing ones by generating more detailed supervisory signals. This approach aims at teaching the re-ranker the reasoning behind relevance. For example, DKS-RAC [58] generates dense knowledge similarity scores to provide the re-ranker with clearer information for evaluating documents. By training on these richer signals, the model can capture more nuanced relationships between a query and a document.

5.2 Filtering

In the post-retrieval stage of RAG systems, filtering and re-ranking are two different steps. Filtering aims at removing documents from the retrieved set that are not relevant or have low quality. This process reduces the size of the document set, which improves the efficiency and effectiveness of the following steps. Re-ranking, on the other hand, arranges the documents that remain in order based on how relevant or useful they are for the task.

One common approach is to use the main language model itself to perform the filtering. This is an efficient design because it does not need a separate external model. For example, Self-RAG [6] uses a self-reflection process where the model generates special "reflection tokens" to evaluate the relevance of retrieved documents and its own generated text. This ensures that only the most suitable documents are kept. A similar idea is used in BlendFilter [160], which also uses the language model as the filter. It assesses and removes less useful documents by applying this filtering to knowledge retrieved from original, externally augmented, and internally augmented queries. Both methods show that the main model's own understanding can be used to clean the retrieved information before generation.

Other methods use different components or operate on smaller pieces of information instead of whole documents. For instance, CRAG [173] uses a "decompose-then-recompose" strategy. It breaks retrieved documents into smaller "knowledge strips" and uses a separate, fine-tuned T5 model to evaluate the relevance of each strip. Only the relevant strips are reassembled to form a refined set of information for the generator. This is different from using the main language model because it uses a specialized model just for filtering. Another approach, RECOMP [171], filters information by generating summaries from retrieved documents. If a document is not relevant, the compressor can generate an empty summary, which effectively filters out that information.

Instead of evaluating each piece of information independently, ETS [146] models evidence selection as a search problem. It uses Monte Carlo Tree Search to find the best combination of sentences that produce a correct answer. This approach evaluates the collective value of sentences working together, not just their individual relevance. These methods show a move toward more fine-grained control, allowing systems to keep useful parts of a document while removing the noisy sections.

Filtering can also happen dynamically during other processes, often guided by the model's confidence. FiD-TF [8], for example, performs "Token Filtering" while the model is generating the answer. It filters out less important tokens based on cross-attention scores, which measure how much attention the model is paying to them. This makes the generation process more efficient. Similarly, CoK [96] uses self-consistency as a signal. It samples several reasoning paths and answers, and only predictions with high consistency are preserved. Questions that do not meet a certain consistency level are processed further, which effectively filters out unreliable reasoning paths.

To create a more dependable filter, some methods combine several criteria. FILCO [166] uses three different strategies to filter at the sentence level: string matching, lexical overlap, and a measure of how much the context improves output likelihood using cross-mutual information. By using these different signals together, the system can more accurately refine the retrieved content. FILCO also trains a context filtering model with these strategies to predict the most useful context at inference time, which helps improve the quality of the final generated output.

6 Generation

6.1 Enhancing

Enhancing methods are strategies aimed at improving the quality and relevance of generated outputs by integrating retrieved content in various ways. These methods differ in how they combine, aggregate, or refine retrieved information, offering multiple approaches to enrich the final output. Broadly, these techniques can be grouped into three categories: enhancing with queries, enhancing with ensemble approaches, and enhancing with feedback loops.

Enhancing with Queries. One common way to integrate external knowledge is to combine the retrieved documents with the user's query before the generation process begins. Implementations of this method differ based on where and how this integration happens. A straightforward approach, seen in In-Context RALM [129], is to place the retrieved documents directly into the input context window along with the original query. This method is simple to implement and works with many existing large language models without needing to change their structure. However, it relies entirely on the model's standard attention mechanism to connect the retrieved information to the query. Its effectiveness is limited by the size of the model's context window, and performance can decrease when the context is very long due to the "lost in the middle" problem, where models often ignore information placed in the middle of a long input [103]. One strategy to address this is context compression. REFRAG [100] shows that much of the retrieved text is not always needed to answer the query. Based on this observation, it compresses parts of the retrieved text into smaller, more compact representations. This action reduces the total length of the input, which greatly lowers the time required to generate the first token, often without a loss in performance. Another strategy involves changing the model's architecture. For instance, RETRO [11] uses a special cross-attention mechanism that allows the model to look at relevant parts of the retrieved documents at different stages of the generation process.

Enhancing with Ensemble. Since a single retrieved document may not contain all the necessary information, many systems retrieve multiple documents. The challenge then becomes how to effectively combine information from these different sources. One strategy is to process all retrieved documents together to form a single, more complete answer. The **Fusion-in-Decoder** (**FiD**) model

[64] does this by encoding each retrieved document separately and then feeding all of them into the decoder at once. This allows the generator to see all the evidence simultaneously and synthesize information across different documents while creating the final response. This approach is useful for building a complete picture from scattered information. An alternative strategy involves treating each retrieved document as an independent source of evidence. In REPLUG [141], the system processes each document and the query separately to generate a potential answer for each document. These separate outputs are then combined, for example, by averaging their probabilities. This method is easier to scale when many documents are retrieved, as each one is handled independently. However, it does not allow the model to compare or combine details from different documents during the generation of a single answer, and it requires more computation because the model must process each document individually. A different strategy avoids the input context entirely. P-RAG [144] puts external knowledge directly into a model's parameters. This is done by turning documents into sets of parameters, called parametric representations, in an offline process. When the model needs to answer a question, it adds the parametric representations from the most relevant documents to its own weights. This approach lowers the computational cost during use, but it needs a large amount of computation beforehand to process all the documents.

Enhancing with Feedback. While processing retrieved information in a single pass is direct, it may not be sufficient for complex queries. Consequently, many approaches incorporate feedback loops to iteratively refine the generated output. These methods can be broadly understood by the nature of their feedback process. One group of methods focuses on improving the collaboration between the retriever and the generator. In these systems, a feedback signal from one component is used to adjust the behavior of another. For example, PRCA [175] places an adapter between the retriever and the generator that learns to refine the retrieved context. It uses feedback from the generator, such as ROUGE-L scores [98], to determine which details in the context are most useful, effectively teaching the retriever what the generator needs. Alternatively, DSP [83] refines the retrieval process itself over multiple steps. The language model generates intermediate queries to build upon previously retrieved information, using a form of programmatic feedback to guide the multi-hop retrieval path toward a better final answer. A second group of methods employs feedback for self-correction, where the language model iteratively refines its own generated text. These approaches enable the model to reflect on its output and make improvements. For instance, Self-mem [21] allows a model to learn from its own past work by creating a "memory pool" from its previous generations. Before generating a new response, the model retrieves information from this memory, which helps it produce more consistent and accurate text over time. Other self-correction techniques focus on verifying the generated information. RECITE [148] prompts the model to produce several pieces of information from its internal knowledge and then applies a self-consistency check to select the most reliable facts for the final answer. This idea of self-verification is also central to the **Chain-of-Verification** (**CoVe**) method [33], where the model first generates a draft response, then formulates and answers questions to check its own claims against retrieved evidence before producing a final, corrected response. This iterative nature, whether through inter-component feedback or self-correction, introduces a tradeoff. While it often leads to higher accuracy and better reasoning on difficult tasks, the multiple steps of retrieval or generation increase response time, making these methods less suitable for applications that require immediate answers.

6.2 Customization

Customization in RAG aims at shaping outputs according to a user's individual needs and personality. This process goes beyond simple text generation by adjusting the retrieval and generation stages to reflect a user's preferences, historical context, and specific requirements. Early methods focused on

using a user's past documents as the primary source for personalization. However, this approach is less effective for new users or those with limited interaction history, a common problem in many real-world systems [7]. To address this, current research explores several strategies for creating more tailored and relevant responses.

One strategy is to enrich a user's profile with additional, related information from external sources. This is particularly useful when a user's own history is sparse. For example, LAPDOG [57] expands a user's persona profile by retrieving external stories that align with their known characteristics. The generator then uses this augmented profile, along with the dialogue history, to produce responses that are more consistent with the user's personality. While this method can provide a richer context for generation, its effectiveness depends on the quality of the retrieved external documents, which may sometimes introduce irrelevant information.

A related strategy moves beyond using only complete documents and instead creates a more detailed user profile from various signals. Some systems use explicit user attributes to guide generation. ERAGent [142], for instance, incorporates stated user preferences, such as environmental consciousness or dietary restrictions, into its personalized reader module to generate responses that reflect the user's values. Other studies create user profiles from stylistic patterns in their writing. For instance, AFCE-RAG [177] improves personalization by providing the model with features such as an author's average sentiment and most frequently used words. The authors also introduce the use of contrastive examples, where documents from different authors are retrieved to help the model better identify a user's unique style by showing what it is not. These approaches show a trend toward building more structured and detailed user representations that capture specific traits rather than just general topics.

Another important trend is to make the retrieval process itself personalized. Instead of using a generic retriever, these methods adapt the retrieval step to each user. ROPG [136] trains a model to select the most suitable retrieval strategy from recency-based, keyword-based, and semantic methods for a given user and query. This selection occurs before generation, ensuring the retrieved documents from the user's own history are highly relevant to the immediate context. Other approaches expand the pool of retrieved documents by looking at similar users. CFRAG [140] uses collaborative filtering to identify users with similar histories and retrieves documents from both the target user and their peers. Similarly, PGraphRAG [7] builds a user-centric knowledge graph to retrieve context from a user's direct history and from "neighboring" users who have interacted with the same items. A key benefit of these collaborative and graph-based methods is their ability to handle the "cold-start" problem, as they can supply meaningful context for users with very little or no personal history.

Finally, some systems focus on dynamic adaptation by updating their understanding of the user in real time. PersonaRAG [185] uses a multi-agent system that continuously analyzes a user's behavior during a session. This allows the system to adjust its responses on the fly, ensuring that the generated content remains aligned with the user's immediate needs and feedback. This constant refinement enables the system to adapt to a user's changing goals within a single interaction, representing a highly responsive form of customization.

7 Evaluation in RAG

Evaluating RAG systems has emerged as a crucial research focus to assess how effectively language models can generate more accurate, relevant, and robust responses by leveraging external knowledge. Given the rising popularity of dialogue-based interactions, much recent work has concentrated on evaluating RAG models' performance on these knowledge-intensive tasks using established metrics like **Exact Match** (**EM**) and F1 scores. These metrics have been applied across a wide array of datasets, including TriviaQA [79], HotpotQA [176], FEVER [152], **Natural Questions**

Table 2. The Comparison of Different RAG Evaluation Frameworks

Evaluation Framework	Aspects	Methods	Metrics	Datasets
RAGAS [37]	Quality of RAG Systems	Context Relevance	Extracted Sentences / Total Sentences	WikiEval[3]
		Answer Relevance	Average Cosine Similarity	
		Faithfulness	Supported Statements / Total Statements	
ARES [135]	Improving RAGAS	Context Relevance	Confidence Intervals	KILT [124] SuperGLUE [159]
		Answer Relevance		
		Answer Faithfulness		
RECALL [104]	Counterfactual Robustness	Response Quality	Accuracy (QA) BLEU [121], ROUGE-L (Generation)	EventKG [45] UJ [55]
		Robustness	Misleading Rate (QA) Mistake Reappearance Rate (Generation)	
RGB [16]	Impact of RAG on LLMs	Noise Robustness	Accuracy	Synthetic
		Negative Rejection	Rejection Rate	
		Information Integration	Accuracy	
		Counterfactual Robustness	Error Detection Rate Error Correction Rate	
MIRAGE [169]	RAG in Medical QA	Zero-Shot Learning	Accuracy	MMLU-Med [51] MedQA-US [75] MedMCQA [119] PubMedQA [77] BioASQ [156]
		Multi-Choice Evaluation		
		Retrieval-Augmented Generation		
		Question-Only Retrieval		
eRAG [137]	Retrieval Quality in RAG	Downstream Task	Accuracy, ROUGE	KILT
		Set-based	Precision, Recall, Hit Rate	
		Ranking	MAP, MRR, NDCG	
UDCG [154]	Retrieval Utility & Distraction	Utility-based Annotation	UDCG	PopQA [112] NoMIRACL [151] NQ [85] TriviaQA [79] BioASQ
BERGEN [131]	Standardizing RAG Experiments	Surface-Based	EM, F1, Precision, Recall	NQ TriviaQA
		Semantic	BEM [13], LLMeval [131]	

[3]https://huggingface.co/datasets/explodinggradients/WikiEval.

(**NQ**) [85], **Wizard of Wikipedia** (**WoW**) [34], and T-REX [36], which are often used to benchmark the effectiveness of retrieval and generation components in knowledge-intensive tasks.

A complete analysis of the different evaluation approaches is extensive enough to be the subject of a dedicated survey article. For a detailed review of RAG evaluation, we refer readers to recent surveys on the topic [39, 179]. In this section, we provide a concise overview by summarizing several widely used evaluation frameworks in Table 2. These frameworks generally assess the two main stages of a RAG pipeline: retrieval and generation.

Retrieval-Based Aspect. The evaluation of the retrieval component examines the quality of the documents that the system retrieves. This is not only about the relevance of the documents to the query but also about their usefulness for the generation step. As shown in Table 2, frameworks like RAGAS and ARES measure aspects such as *Context Relevance*, which helps determine if the retrieved information is precise and supports the final answer. Other frameworks like eRAG [137] assess retrieval quality by measuring the impact on the performance of the downstream generation task. While humans scan results sequentially, LLMs process retrieved documents holistically. These models are sensitive to distractors, which are irrelevant but semantically related documents that degrade performance. To address this issue, UDCG [154] measures the positive contribution of relevant documents and the negative impact of distracting ones, thereby achieving a higher correlation with the accuracy of the final answer compared to traditional metrics.

Generation-Based Aspect. The evaluation of the generation component focuses on the quality of the final text produced by the system. The goal is to measure whether the response is accurate, coherent, and well-supported by the retrieved information. Table 2 lists several aspects, such as

Answer Relevance, which checks whether the output correctly addresses the query, and *Faithfulness*, which verifies that the generated text is factually consistent with the source documents [37]. Frameworks also assess the system's robustness against different challenges, such as its ability to handle misleading information or reject questions that cannot be answered from the provided context [16, 104].

8 Challenges and Future Directions

The evolving landscape of RAG systems faces significant challenges that impact the quality of generated outputs, system efficiency, and the handling of long contexts. As these systems become more prevalent across a range of applications, addressing these challenges is essential for improving their effectiveness and scalability.

8.1 The Role and Selection of Retrieval Models

Table 3 presents a detailed analysis of the RAG studies discussed in this article. The analysis shows that the majority of these studies utilize external data sources to enrich the context of LLMs. A preference for multi-hop over single-hop retrieval was noted, indicating that iterative search rounds generally yield superior results. Furthermore, most methods employ dense retrieval to secure higher-quality candidate documents. Compared to modifying datasets in the pre-retrieval stage, more studies focus on manipulating the query to improve retrieval performance. Additionally, there is a significant emphasis on optimizing the retrieval phase, highlighting its crucial role in the research. However, there seems to be a scarcity of studies concentrating on customization in the generation stage, pointing to this as a potential area for future exploration. Overall, while the goal of RAG is to enhance the response quality of LLMs, greater efforts have been directed toward improving retrieval aspects.

A central challenge in RAG is the selection of the retrieval model, as it directly influences the quality of the information passed to the generator [192]. The choice between traditional retrieval models, such as BM25, and modern neural IR models is not straightforward and involves important tradeoffs. As shown by the studies summarized in Table 4, many systems still use traditional retrievers such as BM25. The main reason for this is efficiency. BM25 is computationally fast and does not require the large resources needed for dense vector indexing and search. In domain-specific applications where terminology is standardized and queries often rely on exact keywords, a BM25 approach can be a highly effective and efficient solution. It provides a strong baseline that is difficult to outperform when semantic understanding is less critical than lexical matching.

On the other hand, neural retrieval models, which include dense and learned sparse approaches, offer a significant advantage in understanding the semantics of a query. These models can identify relevant documents even when there is no direct keyword overlap, which is a common limitation of BM25. This ability to capture user intent and context often leads to higher-quality retrieval results [188]. As shown in experiments from eRAG [137], improving the quality of the retriever with advanced neural models and re-ranking consistently leads to better performance in the final generation task. This confirms that in many cases, especially for open-domain queries, neural models can outperform standard retrieval methods.

The decision, therefore, depends on the specific application's requirements. A system that needs to answer a wide range of general questions may benefit greatly from a powerful neural retriever, while an internal search engine for a specialized corpus might be better served by the speed and simplicity of BM25 [46]. This balance between performance and efficiency is a key consideration. A promising direction for future research is the development of retrieval models designed specifically for RAG. The current landscape shows a limited use of IR-focused large language models as retrievers,

Table 3. The Comprehensive Summary of RAG Studies

Research	Year	Retrieval Source		Multi-hop	Training	Pre-Retrieval		Retrieval		Post-Retrieval		Generation	
		Internal	External			Indexing	Data Modification	Query Manipulation	Search and Ranking	Re-Ranking	Filtering	Enhancing	Customization
REALM [48]	2020		✓		✓	✓			✓				
kNN-LMs [82]	2020	✓	✓			✓			✓			✓	
RAG [91]	2020		✓		✓	✓			✓				
FiD [64]	2021		✓						✓			✓	
Webgpt [115]	2021		✓	✓	✓	✓		✓	✓		✓	✓	
Re2G [44]	2022	✓		✓	✓					✓			
RETRO [11]	2022		✓	✓	✓	✓			✓				
DSP [83]	2022		✓	✓				✓		✓		✓	
CoK [96]	2023		✓	✓				✓		✓			
IRCOT [155]	2023		✓	✓				✓				✓	
ITRG [38]	2023	✓	✓	✓					✓			✓	
PKG [107]	2023	✓											
RA-DIT [101]	2023		✓	✓	✓		✓		✓			✓	
Self-RAG [6]	2023		✓		✓						✓		
SURGE [80]	2023	✓							✓				
FiD-TF [8]	2023		✓							✓	✓		
PRCA [175]	2023		✓		✓				✓			✓	
REPLUG [141]	2023		✓		✓							✓	
AAR [183]	2023		✓		✓				✓				
Query2doc [162]	2023	✓						✓					
Step-Back [189]	2023		✓	✓				✓					
ITER-RETGEN [139]	2023		✓	✓					✓	✓			
RECITE [148]	2023	✓		✓	✓		✓					✓	
PROMPTAGATOR [30]	2023	✓		✓				✓		✓	✓		
UPRISE [20]	2023	✓		✓	✓		✓		✓			✓	
GENREAD [182]	2023	✓					✓					✓	
LAPDOG [57]	2023		✓		✓			✓	✓	✓		✓	✓
KnowledGPT [165]	2023		✓	✓			✓	✓					
Selfmem [21]	2023		✓	✓	✓					✓		✓	
MEMWALKER [15]	2023		✓			✓			✓			✓	
RECOMP [171]	2023		✓		✓						✓		
Rewrite-Retrieve-Read [109]	2023		✓		✓			✓					
Atlas [109]	2023		✓	✓	✓	✓			✓	✓			
DKS-RAC [58]	2023		✓	✓	✓					✓	✓		
In-Context RALM [129]	2023		✓							✓			
Fid-light [53]	2023		✓	✓						✓			
FLARE [73]	2023		✓					✓	✓				
Chameleon [71]	2023		✓		✓	✓			✓				
ERAGent [142]	2024	✓	✓		✓			✓	✓		✓	✓	✓
PipeRAG [72]	2024		✓	✓	✓	✓						✓	
GenRT [172]	2024	✓			✓					✓	✓		
PersonaRAG [185]	2024	✓	✓	✓	✓			✓	✓	✓		✓	✓
CRAG [173]	2024	✓	✓		✓		✓			✓	✓	✓	
IMRAG [174]	2024		✓	✓	✓			✓		✓		✓	
AiSAQ [149]	2024	✓				✓			✓	✓			
ROPG [136]	2024	✓			✓				✓			✓	✓
RQ-RAG [14]	2024		✓	✓	✓			✓		✓			
PlanRAG [89]	2024		✓	✓				✓		✓		✓	
RARG [184]	2024		✓		✓		✓		✓			✓	
DRAGIN [145]	2024		✓	✓				✓	✓			✓	
LRUS-CoverTree [108]	2024	✓			✓	✓			✓	✓			
LightRAG [47]	2024		✓	✓		✓		✓	✓			✓	
REFRAG [100]	2025		✓		✓							✓	
PGraphRAG [7]	2025		✓	✓		✓			✓			✓	✓
UniRAG [94]	2025		✓		✓			✓	✓				
ETS [146]	2025		✓		✓						✓		
P-RAG [144]	2025		✓		✓		✓					✓	
AFCE-RAG [177]	2025		✓				✓		✓				✓
CFRAG [140]	2025		✓		✓		✓		✓	✓		✓	✓

A ✓ in the "Multi-hop" column signifies that the research involves multiple search rounds. Similarly, a ✓ in the "Training" column indicates that the study included training phases. It is important to note that in this context, "Training" encompasses both initial model training and fine-tuning processes.

which points to an opportunity to create new models that combine the semantic power of neural approaches with the efficiency of traditional methods.

8.2 Retrieval Quality

The quality of retrieval is fundamental to any effective RAG system, directly influencing the relevance and accuracy of the generated content [52, 137, 163, 190]. Current retrieval methods, however, frequently struggle with issues like noise, irrelevant documents, and fragmented information, all of which compromise the generation process.

Noise Robustness. Irrelevant or misleading documents within the retrieved set can introduce noise, leading to hallucinations or unreliable answers. This challenge highlights the need for more

Table 4. The Summary of Retrievers and Generators

Research	Year	Retriever	Generator
REALM [48]	2020	BERT [32]	Transformers [157]
kNN-LMs [82]	2020	FAISS [78]	Transformers
RAG [91]	2020	DPR [81]	BART-Large [90]
FiD [64]	2021	BM25 [134], DPR	T5 [128]
Webgpt [115]	2021	Bing	GPT-3 [12]
Re2G [44]	2022	BM25, DPR	BART
RETRO [11]	2022	BERT	Transformer
DSP [83]	2022	ColBERTv2 [84]	GPT-3.5 (text-davinci-002)
CoK [96]	2023	LLaMA2-7B [153], ChatGPT (gpt-3.5-turbo-0613)	ChatGPT (gpt-3.5-turbo-0613)
IRCOT [155]	2023	BM25	GPT-3 (code-davinci-002), Flan-T5 [25]
ITRG [38]	2023	Atlas [109]	LLaMA-33B
PKG [107]	2023	LLaMA-7B	InstructGPT-3.5 (text-davinci-002) [118]
RA-DIT [101]	2023	DRAGON+ [99]	LLaMA
Self-RAG [6]	2023	Contriever [63]	LLaMA2 (7B and 13B) , GPT-4 [1]
SURGE [80]	2023	Graph Neural Networks (GNN) [49]	Transformers
FiD-TF [8]	2023	BM25, SBERT [133]	T5
PRCA [175]	2023	BM25, DPR, Contriver, SimCSE [42], SBERT	T5, Phoenix-7B [19], Vicuna-7B [23], ChatGLM [35], GPT-3.5
REPLUG [141]	2023	Contriever	GPT-3
AAR [183]	2023	ANCE [170], Contriever	Flan-T5, InstructGPT
Query2doc [162]	2023	BM25, DPR	GPT-3 (text-davinci-003)
Step-Back [189]	2023	PaLM-2L [24]	PaLM-2L, GPT-4
ITER-RETGEN [139]	2023	Contriever	InstructGPT (text-davinci-003), LLaMA2
RECITE [148]	2023		PaLM, UL2 [150], OPT [186], Codex [18]
PROMPTAGATOR [30]	2023	T5	FLAN
UPRISE [20]	2023	GPT-Neo-2.7B [10]	BLOOM-7.1B [167], OPT-66B, GPT-3-175B
GENREAD [182]	2023		InstructGPT
LAPDOG [57]	2023	Contriever	T5
KnowledGPT [165]	2023		GPT-4
Selfmem [21]	2023	BM25	XGLM [102], XLM-Rbase [26]
MEMWALKER [15]	2023	LLaMA2	LLaMA2
RECOMP [171]	2023	BM25	T5-Large
Rewrite-Retrieve-Read [109]	2023	Bing	T5-Large, ChatGPT (gpt-3.5-turbo), Vicuna-13B
Atlas [109]	2023	Contriever	T5
DKS-RAC [58]	2023	DPR	BART
In-Context RALM [129]	2023	BM25, BERT-base, Contriever, Spider [130]	GPT-2, GPT-Neo, GPT-J [40], OPT, and LLaMA
Fid-light [53]	2023	GTR-Base [116]	T5
FLARE [73]	2023	BM25, Bing	GPT-3.5 (text-davinci-003)
Chameleon [71]	2023	ChamVS [71]	ChamLM [71]
ERAGent [142]	2024	Bing	GPT-3.5, Falcon 1B [122]
PipeRAG [72]	2024	SBERT	RETRO [11]
GenRT [172]	2024	LambdaMart [2]	
PersonaRAG [185]	2024	BM25	GPT-3.5
CRAG [173]	2024	Contriever	LLaMA2
IMRAG [174]	2024	DPR	Vicuna-7B
AiSAQ [149]	2024	DiskANN [120]	
ROPG [136]	2024	BM25, Contriever	FlanT5-XXL
RQ-RAG [14]	2024	DuckDuckGo[4]	LLaMA2-7B
PlanRAG [89]	2024	GPT-4	GPT-4

(Continued)

sophisticated filtering and context-aware retrieval methods that can better differentiate relevant from irrelevant content. However, Cuconasu et al. [28] present an interesting perspective by showing that, under certain conditions, the inclusion of irrelevant documents can enhance overall accuracy. This finding challenges conventional retrieval strategies and suggests the potential for developing specialized approaches that strategically integrate noise within the retrieval process.

Table 4. Continued

RARG [184]	2024	BM25, E5 [161]	LLaMA2-7B
DRAGIN [145]	2024	BM25, SGPT [114]	LLaMA2 (7B and 13B), Vicuna-13B
LRUS-CoverTree [108]	2024	k-MIPS	
LightRAG [47]	2024	nano-VectorDB[5]	GPT-4o-mini
REFRAG [100]	2025	DRAGON+, RoBERTa	Llama-2-7B, Llama-2-13B, Llama-3.1-8B, Llama-3.2-3B
PGraphRAG [7]	2025	BM25, Contriever	Llama-3.1-8B, GPT-4o-mini
UniRAG [94]	2025	Contriever	Llama-3-8B, Llama-3-70B, GPT-4o-mini
ETS [146]	2025	BGE [168]	Qwen2.5-14B-Instruct-1M, Qwen2.5-72B
P-RAG [144]	2025	BM25	Qwen2.5-1.5B, Llama-3.2-1B, Llama-3-8B
AFCE-RAG [177]	2025	Contriever	Gemma-2-27B
CFRAG [140]	2025	BGE	Llama3-8B, Qwen2-7B

The retrieval models and pre-trained language models explicitly mentioned in these studies have been recorded.
[4]https://duckduckgo.com.
[5] https://github.com/gusye1234/nano-vectordb.

Negative Rejection. When retrieval fails to return relevant results, models often attempt to generate responses regardless, increasing the risk of incorrect outputs. This issue is particularly problematic when queries are poorly expressed or lack sufficient context, making it difficult for retrieval models to surface relevant documents. Techniques like generating a pseudo-document that captures the query's essence, as demonstrated by HyDE [41], can help bridge this gap. By allowing retrieval systems to find more relevant documents even from suboptimal queries, HyDE improves retrieval accuracy, albeit with a tradeoff in computational cost. Another way to handle unclear queries is through interactive refinement. SageRAG [158] uses a human-in-the-loop mechanism to rewrite queries to avoid the drift that can happen in fully automated systems. This method evaluates the initial query and produces multiple improved versions for the user to review. The user then selects the most accurate version before retrieval begins. This curation ensures the search process stays focused on the user's actual intent, which leads to better document relevance and more factual responses. Future work could focus on making these refinement processes faster to balance accuracy with lower latency and less effort for the user.

Information Integration. Complex queries often require synthesizing information from multiple documents, yet fragmented or conflicting information can result in incoherent or incomplete answers. Pre- and post-retrieval techniques play a critical role in addressing this challenge. Enhancing retrieval granularity and incorporating techniques like entity-level retrieval and re-ranking can improve the cohesiveness of retrieved documents. A compelling application of entity-level retrieval is bridging substantial semantic gaps across entirely different disciplines. Cui et al. [29] demonstrate that integrating a unified cross-domain knowledge graph into the RAG pipeline successfully aligns engineering problems with biological principles. Their system uses structured entity-relationship modeling and sampling algorithms to find cross-domain insights that standard retrieval methods often overlook. However, these advanced integration methods face practical limitations. Zhu et al. [192] note that many post-retrieval methods rely heavily on calling LLM APIs, which incurs significant costs. Furthermore, Cuconasu et al. [27] demonstrate that complex passage reordering strategies fail to outperform random shuffling in realistic scenarios. This occurs because state-of-the-art retrievers frequently introduce "hard distractors" that degrade performance regardless of position. Therefore, mitigating these errors requires fundamental improvements in retrieval precision and distraction robustness rather than relying on prompt-based positioning optimizations. Exploring alternatives such as knowledge distillation to lightweight models could offer more scalable solutions, making advanced retrieval strategies more practical in online settings.

Recent research highlights the development of generative models for search as a promising direction for improving retrieval quality. Models like GERE [17] and PARADE [92] enhance document re-ranking and fact verification by directly generating relevant document titles or evidence sentences. Fine-tuning pre-trained models like RankT5 [193] for ranking-specific tasks has also demonstrated potential in boosting out-of-domain performance, which is crucial for generalizing RAG systems across diverse contexts.

8.3 System Efficiency

While retrieval-augmented generation improves the quality of model outputs, it introduces additional steps that can increase response time and computational needs. Making these systems efficient, especially for real-time use, is a significant research area. The main challenges relate to the speed of the retrieval process, the cost of processing long documents, and the overall design of the RAG workflow.

The Retrieval Latency Bottleneck. The retrieval step itself is often the primary source of delay in a RAG pipeline. Performing an exact nearest neighbor search to find the most relevant documents requires scanning an entire database of vectors, a process that can consume up to 97% of the total inference time in some systems [127]. A common alternative is to use approximate nearest neighbor search methods, which are faster but less accurate. This creates a difficult tradeoff. A faster, lower-quality search may return documents of lower relevance, requiring the system to retrieve a larger number of candidates to find the correct information. This larger set of documents then increases the processing load on the generator model, which can negate the initial speed gains from the approximate search and sometimes even increase the total end-to-end latency [127].

Context Compression and Selection. The documents retrieved as context add a substantial number of tokens to the input of the large language model. This increased input length slows down the generation process, as the computational requirements of the self-attention mechanism grow with the length of the input sequence [22, 132]. To address this, researchers have developed methods for context compression and selection. These approaches can be grouped into two main categories. The first category includes lexical methods that reduce the context by selecting only the most important text. This is sometimes called sparse context selection. For example, some systems identify and extract only the most essential sentences from a document, a strategy referred to as Focus Mode [95]. Other methods, like Sparse RAG, build this selection mechanism directly into the generation process. Sparse RAG assesses each retrieved document and allows the generator to selectively use only the caches of the most relevant documents, which reduces the computational load during decoding and avoids the need for a separate filtering model [191]. Similarly, techniques like LLMLingua and RECOMP use different ways to identify and remove less important tokens from the context before it is sent to the generator [22, 164]. The second category includes embedding-based methods, which achieve a more extreme form of compression by replacing the text of the documents with a small number of dense vectors. For instance, the xRAG method uses the document embeddings created for the retrieval step as a direct input to the language model, completely removing the need for the textual documents [22]. Another approach, COCOM, compresses long documents into a few Context Embeddings that are then used by the generator [132]. These compression techniques have shown large efficiency improvements. xRAG was able to reduce the total number of floating-point operations by a factor of 3.53, and COCOM achieved an inference speed-up of up to 5.69 times [22, 132].

Architectural and Workflow Optimizations. Beyond compressing the context, other strategies focus on optimizing the overall RAG workflow. One approach is adaptive retrieval, where the

system first determines if retrieval is necessary for a given query. For simple questions where the language model already has the knowledge, the retrieval step can be skipped entirely, saving time and resources [143]. Another advanced technique is pipeline parallelism, as introduced in PipeRAG [72]. In a standard RAG system, the retrieval and generation processes occur sequentially, meaning one part of the system is often idle. PipeRAG allows these two processes to run at the same time. It achieves this by starting the next retrieval task using a slightly older, or stale, version of the query context. This pre-fetches the necessary documents so they are ready when the generator needs them, effectively hiding the time it takes to retrieve them [72]. By organizing the workflow more efficiently, these architectural improvements help reduce the overall latency of the RAG system.

8.4 Long-Context RAG

The growth of context windows in LLMs has led to new questions about the role of RAG [97, 181]. A common assumption is that providing more retrieved documents to an LLM with a large context capacity should always improve answer quality. However, studies demonstrate otherwise. Instead, as the number of retrieved passages increases, the performance of many long-context LLMs first improves and then declines [74]. This behavior is often called an "inverted U-shaped" performance curve. A main reason for this is the introduction of hard negatives, which are documents that seem related to the query but do not contain the correct answer. These documents can distract the model and lower the quality of the generated response. For example, tests involving the OpenScholar system [5] demonstrate that providing the standard Llama 3.1-8B model with more than ten passages results in lower factual accuracy and weaker citation quality. These findings indicate that smaller models are easily distracted by irrelevant text and struggle to process large amounts of information without specific training.

Positional Bias and Information Quality. A key reason for the performance drop in long contexts is a problem known as the "lost in the middle" phenomenon [74]. LLMs often pay more attention to information at the beginning and end of a long input sequence, while information in the middle gets less attention. As a RAG system retrieves more documents to improve recall, important information may get placed in this middle section and be overlooked by the generator. This positional bias shows that simply increasing the amount of retrieved text is not an effective strategy. In addition to positional bias, RAG systems face challenges with the quality of the retrieved information itself. Traditional text chunking methods often break long documents into disconnected pieces, which can disrupt the document's natural structure and make it difficult for the model to understand the full context [105]. This fragmentation can lead to incomplete information being passed to the generator.

Advanced Architectures for Long Documents. To address these challenges, researchers are creating new RAG methods designed specifically for long documents, which support RAG's value in the era of long-context models. One effective and simple technique is to reorder the retrieved documents before they are passed to the generator. Instead of sorting documents only by their relevance score, this method places the most relevant documents at the start and end of the input context. This approach works with the model's natural bias, ensuring that critical information is in the high-attention zones. Another strategy is **Order-Preserve RAG** (**OP-RAG**) [181], which demonstrates that keeping the original order of retrieved text chunks helps the model process information in a more natural sequence, thereby improves performance when many chunks are retrieved. Other methods focus on improving how documents are processed after retrieval. For example, LongRefiner models the hierarchical structure of a long document, such as its sections and subsections, to filter out unnecessary information in a structured way before generation [76].

Hybrid Systems. More fundamental changes to the RAG architecture are also being explored. Some methods address the problem of context fragmentation without using traditional chunking. Landmark Embedding, for example, keeps the long document complete and inserts special landmark tokens to generate high-quality representations for smaller parts of the text, like sentences or paragraphs [105]. Other systems change where retrieval happens in the generation process. RetroLM performs retrieval on the model's internal KV cache instead of the raw text [106]. This approach makes the system less sensitive to retrieval errors and more efficient because it avoids processing the same text multiple times. MemoRAG takes inspiration from how humans think by first creating a compressed "global memory" of a long document. It then uses this memory to produce internal clues that guide the retriever, which is helpful for tasks that do not have a clear, searchable query [125]. These approaches show that RAG is evolving to handle the special demands of long contexts.

Finally, the tradeoff between performance and cost has led to hybrid systems that combine RAG and native long-context processing. Direct comparisons show that using the full long context can sometimes outperform RAG, but at a much higher computational cost [97]. RAG's main benefit in these cases is its efficiency, because it processes much less text. To balance these factors, the SELF-ROUTE method uses a dynamic strategy [97]. The system first tries to answer a query with the efficient RAG approach. If the model finds that the retrieved text is not enough, it then switches to processing the full long context. This method combines the cost-saving benefits of RAG with the power of full-context processing, showing how RAG and long-context LLMs can work together as parts of a larger, more flexible system.

9 Conclusions

In this article, we have presented a comprehensive framework for understanding the RAG domain, highlighting its significance in enhancing the capabilities of LLMs. Through a structured overview of RAG, categorizing various methods, and an in-depth analysis of its core technologies and evaluation methods, this study illuminates the path for future research. It identifies crucial areas for improvement and outlines potential directions for advancing RAG applications, especially in textual contexts. This survey aims at elucidating the core concepts of the RAG field from a retrieval perspective, and it is intended to facilitate further exploration and innovation in the reliable retrieval and generation of information.

Acknowledgments

All the work was done when the first author was a PhD student at the Information Retrieval and Knowledge Management Research Lab, York University, Canada. The authors gratefully appreciate the anonymous reviewers and editors for their valuable comments and constructive suggestions that greatly helped to improve the quality of the article.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, Red Avila, Igor Babuschkin, et al. 2023. GPT-4 technical report. arXiv:2303.08774. Retrieved from https://arxiv.org/abs/2303.08774

[2] Qingyao Ai, Keping Bi, Jiafeng Guo, and W. Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *Proceedings of the 41st International ACM SIGIR Conference on Research and Development in Information Retrieval*. 135–144.

[3] Alexandr Andoni, Piotr Indyk, Huy L. Nguyen, and Ilya P. Razenshteyn. 2014. Beyond locality-sensitive hashing. In *Proceedings of the 2014 Annual ACM-SIAM Symposium on Discrete Algorithms*. 1018–1028.

[4] Sunil Arya, David M. Mount, Nathan S. Netanyahu, Ruth Silverman, and Angela Y. Wu. 1998. An optimal algorithm for approximate nearest neighbor searching fixed dimensions. *Journal of the ACM* 45, 6 (1998), 891–923.

[5] Akari Asai, Jacqueline He, Rulin Shao, Weijia Shi, Amanpreet Singh, Joseph Chee Chang, Kyle Lo, Luca Soldaini, Sergey Feldman, Mike D'Arcy, David Wadden, Matt Latzke, Jenna Sparks, Jena D. Hwang, Varsha Kishore, Minyang Tian, Pan Ji, Shengyan Liu, Hao Tong, Bohao Wu, Yanyu Xiong, Luke Zettlemoyer, Graham Neubig, Daniel S. Weld, Doug Downey, Wen-tau Yih, Pang Wei Koh, and Hannaneh Hajishirzi. 2026. Synthesizing scientific literature with retrieval-augmented language models. *Nature* 650, 8103 (2026), 1–7.

[6] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. 2024. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *Proceedings of the International Conference on Learning Representations*. 1–30.

[7] Steven Au, Cameron J. Dimacali, Ojasmitha Pedirappagari, Namyong Park, Franck Dernoncourt, Yu Wang, Nikos Kanakaris, Hanieh Deilamsalehy, Ryan A. Rossi, and Nesreen K. Ahmed. 2025. Personalized graph-based retrieval for large language models. In *Proceedings of the 39th Pacific Asia Conference on Language, Information and Computation*. 678–698.

[8] Moshe Berchansky, Peter Izsak, Avi Caciularu, Ido Dagan, and Moshe Wasserblat. 2023. Optimizing retrieval-augmented reader models via token elimination. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 1506–1524.

[9] Michele Bevilacqua, Giuseppe Ottaviano, Patrick Lewis, Scott Yih, Sebastian Riedel, and Fabio Petroni. 2022. Autoregressive search engines: Generating substrings as document identifiers. In *Proceedings of the Advances in Neural Information Processing Systems* 31668–31683.

[10] Sidney Black, Stella Biderman, Eric Hallahan, Quentin Anthony, Leo Gao, Laurence Golding, Horace He, Connor Leahy, Kyle McDonell, Jason Phang, Michael Pieler, Usvsn Sai Prashanth, Shivanshu Purohit, Laria Reynolds, Jonathan Tow, Ben Wang, and Samuel Weinbach. 2022. GPT-NeoX-20B: An open-source autoregressive language model. In *Proceedings of the BigScience Workshop on Challenges and Perspectives in Creating Large Language Models*. ACL, 95–136.

[11] Sebastian Borgeaud, Arthur Mensch, Jordan Hoffmann, Trevor Cai, Eliza Rutherford, Katie Millican, George Bm Van Den Driessche, Jean-Baptiste Lespiau, Bogdan Damoc, Aidan Clark, Diego De Las Casas, Aurelia Guy, Jacob Menick, Roman Ring, Tom Hennigan, Saffron Huang, Loren Maggiore, Chris Jones, Albin Cassirer, Andy Brock, Michela Paganini, Geoffrey Irving, Oriol Vinyals, Simon Osindero, Karen Simonyan, Jack Rae, Erich Elsen, and Laurent Sifre. 2022. Improving language models by retrieving from trillions of tokens. In *Proceedings of the International Conference on Machine Learning*. 2206–2240.

[12] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, et al. 2020. Language models are few-shot learners. In *Proceedings of the Advances in Neural Information Processing Systems*. 1877–1901.

[13] Jannis Bulian, Christian Buck, Wojciech Gajewski, Benjamin Börschinger, and Tal Schuster. 2022. Tomayto, tomahto. Beyond token-level answer equivalence for question answering evaluation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 291–305.

[14] Chi-Min Chan, Chunpu Xu, Ruibin Yuan, Hongyin Luo, Wei Xue, Yike Guo, and Jie Fu. 2024. RQ-RAG: Learning to refine queries for retrieval augmented generation. arXiv:2404.00610. Retrieved from https://arxiv.org/abs/2404.00610

[15] Howard Chen, Ramakanth Pasunuru, Jason Weston, and Asli Celikyilmaz. 2023. Walking down the memory maze: Beyond context limit through interactive reading. arXiv:2310.05029. Retrieved from https://arxiv.org/abs/2310.05029

[16] Jiawei Chen, Hongyu Lin, Xianpei Han, and Le Sun. 2024. Benchmarking large language models in retrieval-augmented generation. In *Proceedings of the AAAI Conference on Artificial Intelligence*. Association for the Advancement of Artificial Intelligence, 17754–17762.

[17] Jiangui Chen, Ruqing Zhang, Jiafeng Guo, Yixing Fan, and Xueqi Cheng. 2022. GERE: Generative evidence retrieval for fact verification. In *Proceedings of the 45th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2184–2189.

[18] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, Alex Ray, Raul Puri, Gretchen Krueger, Michael Petrov, Heidy Khlaaf, Girish Sastry, Pamela Mishkin, Brooke Chan, Scott Gray, Nick Ryder, Mikhail Pavlov, Alethea Power, Lukasz Kaiser, Mohammad Bavarian, Clemens Winter, Philippe Tillet, Felipe Petroski Such, Dave Cummings, Matthias Plappert, Fotios Chantzis, Elizabeth Barnes, Ariel Herbert-Voss, William Hebgen Guss, Alex Nichol, Alex Paino, Nikolas Tezak, Jie Tang, Igor Babuschkin, Suchir Balaji, Shantanu Jain, William Saunders, Christopher Hesse, Andrew N. Carr, Jan Leike, Josh Achiam, Vedant Misra, Evan Morikawa, Alec Radford, Matthew Knight, Miles Brundage, Mira Murati, Katie Mayer, Peter Welinder, Bob McGrew, Dario Amodei, Sam McCandlish, Ilya Sutskever, and Wojciech Zaremba. 2021. Evaluating large language models trained on code. arXiv:2107.03374. Retrieved from https://arxiv.org/abs/2107.03374

[19] Zhihong Chen, Feng Jiang, Junying Chen, Tiannan Wang, Fei Yu, Guiming Chen, Hongbo Zhang, Juhao Liang, Chen Zhang, Zhiyi Zhang, Jianquan Li, Xiang Wan, Benyou Wang, and Haizhou Li. 2023. Phoenix: Democratizing ChatGPT across languages. arXiv:2304.10453. Retrieved from https://arxiv.org/abs/2304.10453

[20] Daixuan Cheng, Shaohan Huang, Junyu Bi, Yuefeng Zhan, Jianfeng Liu, Yujing Wang, Hao Sun, Furu Wei, Weiwei Deng, and Qi Zhang. 2023. UPRISE: Universal prompt retrieval for improving zero-shot evaluation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 12318–12337.

[21] Xin Cheng, Di Luo, Xiuying Chen, Lemao Liu, Dongyan Zhao, and Rui Yan. 2023. Lift yourself up: Retrieval-augmented text generation with self-memory. In *Proceedings of the Advances in Neural Information Processing Systems*. 43780–43799.

[22] Xin Cheng, Xun Wang, Xingxing Zhang, Tao Ge, Si-Qing Chen, Furu Wei, Huishuai Zhang, and Dongyan Zhao. 2024. xRAG: Extreme context compression for retrieval-augmented generation with one token. In *Proceedings of the Advances in Neural Information Processing Systems*. 109487–109516.

[23] Wei-Lin Chiang, Zhuohan Li, Zi Lin, Ying Sheng, Zhanghao Wu, Hao Zhang, Lianmin Zheng, Siyuan Zhuang, Yonghao Zhuang, Joseph E. Gonzalez, Ion Stoica, and Eric P. Xing. 2023. Vicuna: An Open-Source Chatbot Impressing GPT-4 with 90%* ChatGPT Quality. Retrieved from https://lmsys.org/blog/2023-03-30-vicuna/

[24] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, Parker Schuh, Kensen Shi, Sasha Tsvyashchenko, Joshua Maynez, Abhishek Rao, Parker Barnes, Yi Tay, Noam Shazeer, Vinodkumar Prabhakaran, Emily Reif, Nan Du, Ben Hutchinson, Reiner Pope, James Bradbury, Jacob Austin, Michael Isard, Guy Gur-Ari, Pengcheng Yin, Toju Duke, Anselm Levskaya, Sanjay Ghemawat, Sunipa Dev, Henryk Michalewski, Xavier Garcia, Vedant Misra, Kevin Robinson, Liam Fedus, Denny Zhou, Daphne Ippolito, David Luan, Hyeontaek Lim, Barret Zoph, Alexander Spiridonov, Ryan Sepassi, David Dohan, Shivani Agrawal, Mark Omernick, Andrew M. Dai, Thanumalayan Sankaranarayana Pillai, Marie Pellat, Aitor Lewkowycz, Erica Moreira, Rewon Child, Oleksandr Polozov, Katherine Lee, Zongwei Zhou, Xuezhi Wang, Brennan Saeta, Mark Diaz, Orhan Firat, Michele Catasta, Jason Wei, Kathy Meier-Hellstern, Douglas Eck, Jeff Dean, Slav Petrov, and Noah Fiedel. 2023. PaLM: Scaling language modeling with pathways. *Journal of Machine Learning Research* 24 (2023), 240:1–240:113.

[25] Hyung Won Chung, Le Hou, Shayne Longpre, Barret Zoph, Yi Tay, William Fedus, Yunxuan Li, Xuezhi Wang, Mostafa Dehghani, Siddhartha Brahma, Albert Webson, Shixiang Shane Gu, Zhuyun Dai, Mirac Suzgun, Xinyun Chen, Aakanksha Chowdhery, Alex Castro-Ros, Marie Pellat, Kevin Robinson, Dasha Valter, Sharan Narang, Gaurav Mishra, Adams Yu, Vincent Zhao, Yanping Huang, Andrew Dai, Hongkun Yu, Slav Petrov, Ed H. Chi, Jeff Dean, Jacob Devlin, Adam Roberts, Denny Zhou, Quoc V. Le, and Jason Wei. 2024. Scaling instruction-finetuned language models. *Journal of Machine Learning Research* 25 (2024), 70:1–70:53.

[26] Alexis Conneau, Kartikay Khandelwal, Naman Goyal, Vishrav Chaudhary, Guillaume Wenzek, Francisco Guzmán, Edouard Grave, Myle Ott, Luke Zettlemoyer, and Veselin Stoyanov. 2020. Unsupervised cross-lingual representation learning at scale. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. ACL, 8440–8451.

[27] Florin Cuconasu, Simone Filice, Guy Horowitz, Yoelle Maarek, and Fabrizio Silvestri. 2025. Do RAG systems really suffer from positional bias?. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 28010–28024.

[28] Florin Cuconasu, Giovanni Trappolini, Federico Siciliano, Simone Filice, Cesare Campagnano, Yoelle Maarek, Nicola Tonellotto, and Fabrizio Silvestri. 2024. The power of noise: Redefining retrieval for RAG systems. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 719–729.

[29] Haoran Cui, Pai Zheng, Mengyang Ren, and Yan Yan. 2026. An LLM-based cross-domain knowledge retrieval augmented generation method for bio-inspired solution design. *Advanced Engineering Informatics* 69 (2026), 104017.

[30] Zhuyun Dai, Vincent Y. Zhao, Ji Ma, Yi Luan, Jianmo Ni, Jing Lu, Anton Bakalov, Kelvin Guu, Keith B. Hall, and Ming-Wei Chang. 2023. Promptagator: Few-shot dense retrieval from 8 examples. In *Proceedings of the International Conference on Learning Representations*. 1–22.

[31] Mayur Datar, Nicole Immorlica, Piotr Indyk, and Vahab S. Mirrokni. 2004. Locality-sensitive hashing scheme based on p-stable distributions. In *Proceedings of the International Symposium on Computational Geometry*. 253–262.

[32] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the Conference of the North*. ACL, 4171–4186.

[33] Shehzaad Dhuliawala, Mojtaba Komeili, Jing Xu, Roberta Raileanu, Xian Li, Asli Celikyilmaz, and Jason Weston. 2024. Chain-of-verification reduces hallucination in large language models. In *Findings of the Association for Computational Linguistics: ACL 2024*. ACL, 3563–3578.

[34] Emily Dinan, Stephen Roller, Kurt Shuster, Angela Fan, Michael Auli, and Jason Weston. 2019. Wizard of wikipedia: Knowledge-powered conversational agents. In *Proceedings of the International Conference on Learning Representations*. 1–18.

[35] Zhengxiao Du, Yujie Qian, Xiao Liu, Ming Ding, Jiezhong Qiu, Zhilin Yang, and Jie Tang. 2022. GLM: General language model pretraining with autoregressive blank infilling. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 320–335.

[36] Hady Elsahar, Pavlos Vougiouklis, Arslen Remaci, Christophe Gravier, Jonathon Hare, Elena Simperl, and Frederique Laforest. 2018. T-REx: A large scale alignment of natural language with knowledge base triples. In *Proceedings of the International Conference on Language Resources and Evaluation*. 3448–3452.

[37] Shahul ES, Jithin James, Luis Espinosa Anke, and Steven Schockaert. 2024. RAGAs: Automated evaluation of retrieval augmented generation. In *Proceedings of the 18th Conference of the European Chapter of the Association for Computational Linguistics, EACL 2024 - System Demonstrations*. ACL, 150–158.

[38] Zhangyin Feng, Xiaocheng Feng, Dezhi Zhao, Maojin Yang, and Bing Qin. 2024. Retrieval-generation synergy augmented large language models. In *Proceedings of the 2024 IEEE International Conference on Acoustics, Speech, and Signal Processing*. IEEE, 11661–11665.

[39] Aoran Gan, Hao Yu, Kai Zhang, Qi Liu, Wenyu Yan, Zhenya Huang, Shiwei Tong, and Guoping Hu. 2025. Retrieval augmented generation evaluation in the era of large language models: A comprehensive survey. arXiv:2504.14891. Retrieved from https://arxiv.org/abs/2504.14891

[40] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, Shawn Presser, and Connor Leahy. 2021. The Pile: An 800GB dataset of diverse text for language modeling. arXiv:2101.00027. Retrieved from https://arxiv.org/abs/2101.00027

[41] Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. 2023. Precise zero-shot dense retrieval without relevance labels. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 1762–1777.

[42] Tianyu Gao, Xingcheng Yao, and Danqi Chen. 2021. SimCSE: Simple contrastive learning of sentence embeddings. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 6894–6910.

[43] Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yi Dai, Jiawei Sun, Meng Wang, and Haofen Wang. 2023. Retrieval-augmented generation for large language models: A survey. arXiv:2312.10997. Retrieved from https://arxiv.org/abs/2312.10997

[44] Michael Glass, Gaetano Rossiello, Md Faisal Mahbub Chowdhury, Ankita Naik, Pengshan Cai, and Alfio Gliozzo. 2022. Re2G: Retrieve, rerank, generate. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACL, 2701–2715.

[45] Simon Gottschalk and Elena Demidova. 2018. EventKG: A multilingual event-centric temporal knowledge graph. In *Proceedings of the Semantic Web: 15th International Conference*. Springer International Publishing, 272–287.

[46] Jiafeng Guo, Yinqiong Cai, Yixing Fan, Fei Sun, Ruqing Zhang, and Xueqi Cheng. 2022. Semantic models for the first-stage retrieval: A comprehensive review. *ACM Transactions on Information Systems* 40, 4 (2022), 1–42.

[47] Zirui Guo, Lianghao Xia, Yanhua Yu, Tu Ao, and Chao Huang. 2025. LightRAG: Simple and fast retrieval-augmented generation. In *Findings of the Association for Computational Linguistics: EMNLP 2025*. 10746–10761.

[48] Kelvin Guu, Kenton Lee, Zora Tung, Panupong Pasupat, and Ming-Wei Chang. 2020. Retrieval augmented language model pre-training. In *Proceedings of the International Conference on Machine Learning*. 3929–3938.

[49] William L. Hamilton. 2020. *Graph Representation Learning*. Morgan & Claypool Publishers.

[50] Micheline Hancock-Beaulieu, Mike Gatford, X. Huang, Stephen E. Robertson, Steve Walker, and P. W. Williams. 1996. Okapi at TREC-5. In *Proceedings of the 5th Text REtrieval Conference, TREC 1996 (NIST Special Publication, Vol. 500-238)*. National Institute of Standards and Technology (NIST).

[51] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. 2021. Measuring massive multitask language understanding. In *Proceedings of the International Conference on Learning Representations*. 1–27.

[52] Enrique HerreraViedma, Gabriella Pasi, Antonio G. LopezHerrera, and Carlos Porcel. 2006. Evaluating the information quality of web sites: A methodology based on fuzzy computing with words. *Journal of the American Society for Information Science and Technology* 57, 4 (2006), 538–549.

[53] Sebastian Hofstätter, Jiecao Chen, Karthik Raman, and Hamed Zamani. 2023. Fid-light: Efficient and effective retrieval-augmented text generation. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1437–1447.

[54] Yucheng Hu and Yuxing Lu. 2024. RAG and RAU: A survey on retrieval-augmented language model in natural language processing. arXiv:2404.19543. Retrieved from https://arxiv.org/abs/2404.19543

[55] Jie Huang, Hanyin Shao, Kevin Chen-Chuan Chang, Jinjun Xiong, and Wen-mei Hwu. 2022. Understanding jargon: Combining extraction and generation for definition modeling. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 3994–4004.

[56] Jimmy X. Huang, Jun Miao, and Ben He. 2013. High performance query expansion using adaptive co-training. *Information Processing and Management* 49, 2 (2013), 441–453.

[57] Qiushi Huang, Shuai Fu, Xubo Liu, Wenwu Wang, Tom Ko, Yu Zhang, and Lilian H. Y. Tang. 2023. Learning retrieval augmentation for personalized dialogue generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2523–2540.

[58] Wenyu Huang, Mirella Lapata, Pavlos Vougiouklis, Nikos Papasarantopoulos, and Jeff Z. Pan. 2023. Retrieval augmented generation with rich answer encoding. *Proceedings of the 13th International Joint Conference on Natural Language Processing and the 3rd Conference of the Asia-Pacific Chapter of the Association for Computational Linguistics, IJCNLP 2023 - Volume 1: Long Papers* 2023 (2023), 1012–1025.

[59] X. Huang and Qinmin Hu. 2009. A bayesian learning approach to promoting diversity in ranking for biomedical information retrieval. In *Proceedings of the 32nd Annual International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 307–314.

[60] X. Huang, Yan Rui Huang, Miao Wen, Aijun An, Yang Liu, and Josiah Poon. 2006. Applying data mining to pseudo-relevance feedback for high performance text retrieval. In *Proceedings of the 6th IEEE International Conference on Data Mining*. IEEE Computer Society, 295–306.

[61] Yizheng Huang and Jimmy X. Huang. 2023. Diversified prior knowledge enhanced general language model for biomedical information retrieval. In *Proceedings of the European Conference on Artificial Intelligence*. 1109–1115.

[62] Yizheng Huang and Jimmy X. Huang. 2024. Exploring ChatGPT for next-generation information retrieval: Opportunities and challenges. *Web Intelligence* 22, 1 (2024), 31–44.

[63] Gautier Izacard, Mathilde Caron, Lucas Hosseini, Sebastian Riedel, Piotr Bojanowski, Armand Joulin, and Edouard Grave. 2022. Unsupervised dense information retrieval with contrastive learning. *Transactions on Machine Learning Research* (2022), 1–21.

[64] Gautier Izacard and Edouard Grave. 2021. Leveraging passage retrieval with generative models for open domain question answering. In *Proceedings of the 16th Conference of the European Chapter of the Association for Computational Linguistics: Main Volume*. ACL, 874–880.

[65] Gautier Izacard, Patrick Lewis, Maria Lomeli, Lucas Hosseini, Fabio Petroni, Timo Schick, Jane Dwivedi-Yu, Armand Joulin, Sebastian Riedel, and Edouard Grave. 2023. Atlas: Few-shot learning with retrieval augmented language models. *Journal of Machine Learning Research* 24 (2023), 251:1–251:43.

[66] Omid Jafari, Preeti Maurya, P. Nagarkar, Khandker Mushfiqul Islam, and Chidambaram Crushev. 2021. A survey on locality sensitive hashing algorithms and their applications. arXiv: 2102.08942. Retrieved from https://arxiv.org/abs/2102.08942

[67] Israt Jahan, Md. Tahmid Rahman Laskar, Chun Peng, and Jimmy X. Huang. 2023. Evaluation of ChatGPT on biomedical tasks: A zero-shot comparison with fine-tuned generative transformers. In *Proceedings of the 22nd Workshop on Biomedical Natural Language Processing and BioNLP Shared Tasks*. ACL, 326–336.

[68] Bernard J. Jansen, Danielle L. Booth, and Amanda Spink. 2009. Patterns of query reformulation during web searching. *Journal of the American Society for Information Science and Technology* 60, 7 (2009), 1358–1371.

[69] H. Jégou, M. Douze, and C. Schmid. 2011. Product quantization for nearest neighbor search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128.

[70] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2023. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 13358–13376.

[71] Wenqi Jiang, Marco Zeller, R. Waleffe, Torsten Hoefler, and Gustavo Alonso. 2023. Chameleon: a heterogeneous and disaggregated accelerator system for retrieval-augmented language models. *Proceedings of the VLDB Endowment* 18, 1 (2023), 42–52.

[72] Wenqi Jiang, Shuai Zhang, Boran Han, Jie Wang, Bernie Wang, and Tim Kraska. 2025. PipeRAG: Fast retrieval-augmented generation via adaptive pipeline parallelism. In *Proceedings of the 31st ACM SIGKDD Conference on Knowledge Discovery and Data Mining*. ACM, 589–600.

[73] Zhengbao Jiang, Frank Xu, Luyu Gao, Zhiqing Sun, Qian Liu, Jane Dwivedi-Yu, Yiming Yang, Jamie Callan, and Graham Neubig. 2023. Active retrieval augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 7969–7992.

[74] Bowen Jin, Jinsung Yoon, Jiawei Han, and Sercan Ö. Arik. 2024. Long-context LLMs meet RAG: Overcoming challenges for long inputs in RAG. In *Proceedings of the International Conference on Learning Representations*. 65721–65759.

[75] Di Jin, Eileen Pan, Nassim Oufattole, Wei-Hung Weng, Hanyi Fang, and Peter Szolovits. 2021. What disease does this patient have? A large-scale open domain question answering dataset from medical exams. *Applied Sciences* 11, 14 (2021), 6421.

[76] Jiajie Jin, Xiaoxi Li, Guanting Dong, Yuyao Zhang, Yutao Zhu, Yongkang Wu, Zhonghua Li, Qi Ye, and Zhicheng Dou. 2025. Hierarchical document refinement for long-context retrieval-augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 3502–3520.

[77] Qiao Jin, Bhuwan Dhingra, Zhengping Liu, William W. Cohen, and Xinghua Lu. 2019. PubMedQA: A dataset for biomedical research question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 2567–2577.

[78] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2021. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 3 (2021), 535–547.

[79] Mandar Joshi, Eunsol Choi, Daniel Weld, and Luke Zettlemoyer. 2017. TriviaQA: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 1601–1611.

[80] Minki Kang, Jin Myung Kwak, Jinheon Baek, and Sung Ju Hwang. 2023. Knowledge graph-augmented language models for knowledge-grounded dialogue generation. arXiv:2305.18846. Retrieved from https://arxiv.org/abs/2305.18846

[81] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense passage retrieval for open-domain question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 6769–6781.

[82] Urvashi Khandelwal, Omer Levy, Dan Jurafsky, Luke Zettlemoyer, and Mike Lewis. 2020. Generalization through memorization: Nearest neighbor language models. In *Proceedings of the International Conference on Learning Representations*. 1–13.

[83] Omar Khattab, Keshav Santhanam, Xiang Lisa Li, David Hall, Percy Liang, Christopher Potts, and Matei Zaharia. 2022. Demonstrate-search-predict: Composing retrieval and language models for knowledge-intensive NLP. arXiv:2212.14024. Retrieved from https://arxiv.org/abs/2212.14024

[84] Omar Khattab and Matei Zaharia. 2020. ColBERT - efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 39–48.

[85] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, Kristina Toutanova, Llion Jones, Matthew Kelcey, Ming-Wei Chang, Andrew M. Dai, Jakob Uszkoreit, Quoc Le, and Slav Petrov. 2019. Natural questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics* 7 (2019), 453–466.

[86] Md. Tahmid Rahman Laskar, M. Saiful Bari, Mizanur Rahman, Md Amran Hossen Bhuiyan, Shafiq Joty, and Jimmy X. Huang. 2023. A systematic study and comprehensive evaluation of ChatGPT on benchmark datasets. In *Findings of the Association for Computational Linguistics: ACL 2023*. ACL, 431–469.

[87] Md. Tahmid Rahman Laskar, Enamul Hoque, and Jimmy X. Huang. 2020. WSL-DS: Weakly supervised learning with distant supervision for query focused multi-document abstractive summarization. In *Proceedings of the 28th International Conference on Computational Linguistics*. International Committee on Computational Linguistics, 5647–5654.

[88] Md. Tahmid Rahman Laskar, Enamul Hoque, and Jimmy X.Huang. 2020. Query focused abstractive summarization via incorporating query relevance and transfer learning with transformer models. In *Advances in Artificial Intelligence - 33rd Canadian Conference on Artificial Intelligence, Canadian AI 2020 Proceedings (Lecture Notes in Computer Science, Vol. 12109)*. Springer, 342–348.

[89] Myeonghwa Lee, Seonho An, and Min-Soo Kim. 2024. PlanRAG: A plan-then-retrieval augmented generation for generative large language models as decision makers. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. ACL.

[90] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Veselin Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. ACL, 7871–7880.

[91] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proceedings of the Advances in Neural Information Processing Systems* 33 (2020), 9459–9474.

[92] Canjia Li, Andrew Yates, Sean MacAvaney, Ben He, and Yingfei Sun. 2024. PARADE: Passage representation aggregation fordocument reranking. *ACM Transactions on Information Systems* 42, 2 (2024), 1–26.

[93] Huayang Li, Yixuan Su, Deng Cai, Yan Wang, and Lemao Liu. 2022. A survey on retrieval-augmented text generation. arXiv:2202.01110. Retrieved from https://arxiv.org/abs/2202.01110

[94] Rui Li, Liyang He, Qi Liu, Zheng Zhang, Heng Yu, Yuyang Ye, Linbo Zhu, and Yu Su. 2025. UniRAG: Unified query understanding method for retrieval augmented generation. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 14163–14178.

[95] Siran Li, Linus Stenzel, Carsten Eickhoff, and Seyed Ali Bahrainian. 2025. Enhancing retrieval-augmented generation: A study of best practices. In *Proceedings of the 31st International Conference on Computational Linguistics* ACL, 6705–6717.

[96] Xingxuan Li, Ruochen Zhao, Yew Ken Chia, Bosheng Ding, Shafiq R. Joty, Soujanya Poria, and Lidong Bing. 2024. Chain-of-knowledge: Grounding large language models via dynamic knowledge adapting over heterogeneous sources. In *Proceedings of the International Conference on Learning Representations*.

[97] Zhuowan Li, Cheng Li, Mingyang Zhang, Qiaozhu Mei, and Michael Bendersky. 2024. Retrieval augmented generation or long-context LLMs? A comprehensive study and hybrid approach. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*.

[98] Chin-Yew Lin. 2004. ROUGE: A package for automatic evaluation of summaries. In *Proceedings of the Text Summarization Branches Out*. ACL, 74–81.

[99] Sheng-Chieh Lin, Akari Asai, Minghan Li, Barlas Oguz, Jimmy Lin, Yashar Mehdad, Wen-tau Yih, and Xilun Chen. 2023. How to train your dragon: Diverse augmentation towards generalizable dense retrieval. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. ACL, 6385–6400.

[100] Xiaoqiang Lin, Aritra Ghosh, B. Low, Anshumali Shrivastava, and Vijai Mohan. 2025. REFRAG: Rethinking RAG based decoding. In *Proceedings of the First Workshop on Multi-Turn Interactions in Large Language Models in Advances in Neural Information Processing Systems* (2025).

[101] Xi Victoria Lin, Xilun Chen, Mingda Chen, Weijia Shi, Maria Lomeli, Richard James, Pedro Rodriguez, Jacob Kahn, Gergely Szilvasy, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2024. RA-DIT: Retrieval-augmented dual instruction tuning. In *Proceedings of the International Conference on Learning Representations (ICLR)*. 1–25.

[102] Xi Victoria Lin, Todor Mihaylov, Mikel Artetxe, Tianlu Wang, Shuohui Chen, Daniel Simig, Myle Ott, Naman Goyal, Shruti Bhosale, Jingfei Du, Ramakanth Pasunuru, Sam Shleifer, Punit Singh Koura, Vishrav Chaudhary, Brian O'Horo, Jeff Wang, Luke Zettlemoyer, Zornitsa Kozareva, Mona Diab, Veselin Stoyanov, and Xian Li. 2022. Few-shot learning with multilingual generative language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL.

[103] Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. 2024. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* 12 (2024), 157–173.

[104] Yi Liu, Lianzhe Huang, Shicheng Li, Sishuo Chen, Hao Zhou, Fandong Meng, Jie Zhou, and Xu Sun. 2023. RECALL: A benchmark for LLMs robustness against external counterfactual knowledge. arXiv:2311.08147. Retrieved from https://arxiv.org/abs/2311.08147

[105] Kun Luo, Zheng Liu, Shitao Xiao, Tong Zhou, Yubo Chen, Jun Zhao, and Kang Liu. 2024. Landmark embedding: A chunking-free embedding method for retrieval augmented long-context large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 3268–3281.

[106] Kun Luo, Zheng Liu, Peitian Zhang, Hongjin Qian, Jun Zhao, and Kang Liu. 2025. Does RAG really perform bad for long-context processing? arXiv:2502.11444. Retrieved from https://arxiv.org/abs/2502.11444

[107] Ziyang Luo, Can Xu, Pu Zhao, Xiubo Geng, Chongyang Tao, Jing Ma, Qingwei Lin, and Daxin Jiang. 2023. Augmented large language models with parametric knowledge guiding. arXiv:2305.04757. Retrieved from https://arxiv.org/abs/2305.04757

[108] Hengzhao Ma, Jianzhong Li, and Yong Zhang. 2024. Reconsidering tree based methods for k-maximum inner-product search: The LRUS-covertree. In *Proceedings of the 2024 IEEE 40th International Conference on Data Engineering*. IEEE.

[109] Xinbei Ma, Yeyun Gong, Pengcheng He, Hai Zhao, and Nan Duan. 2023. Query rewriting in retrieval-augmented large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 5303–5315.

[110] Xueguang Ma, Liang Wang, Nan Yang, Furu Wei, and Jimmy Lin. 2024. Fine-tuning LLaMA for multi-stage text retrieval. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2421–2425.

[111] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836.

[112] Alex Mallen, Akari Asai, Victor Zhong, Rajarshi Das, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. When not to trust language models: Investigating effectiveness of parametric and non-parametric Memories. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 9802–9822.

[113] Christopher D. Manning, Prabhakar Raghavan, and Hinrich Schütze. 2008. *Introduction to Information Retrieval*. Cambridge University Press.

[114] Niklas Muennighoff. 2022. SGPT: GPT sentence embeddings for semantic search. arXiv:2202.08904. Retrieved from https://arxiv.org/abs/2202.08904

[115] Reiichiro Nakano, Jacob Hilton, Suchir Balaji, Jeff Wu, Long Ouyang, Christina Kim, Christopher Hesse, Shantanu Jain, Vineet Kosaraju, William Saunders, Xu Jiang, Karl Cobbe, Tyna Eloundou, Gretchen Krueger, Kevin Button, Matthew Knight, Benjamin Chess, and John Schulman. 2021. WebGPT: Browser-assisted question-answering with human feedback. arXiv: 2112.09332. Retrieved from https://arxiv.org/abs/2112.09332

[116] Jianmo Ni, Chen Qu, Jing Lu, Zhuyun Dai, Gustavo Hernandez Abrego, Ji Ma, Vincent Zhao, Yi Luan, Keith Hall, Ming-Wei Chang, and Yinfei Yang. 2022. Large dual encoders are generalizable retrievers. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 9844–9855.

[117] Rodrigo Nogueira, Zhiying Jiang, Ronak Pradeep, and Jimmy Lin. 2020. Document ranking with a pretrained sequence-to-sequence model. In *Findings of the Association for Computational Linguistics: EMNLP 2020*. ACL.

[118] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll L. Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. 2022. Training language models to follow instructions with human feedback. In *Proceedings of the Advances in Neural Information Processing Systems* 27730–27744.

[119] Ankit Pal, Logesh Kumar Umapathi, and Malaikannan Sankarasubbu. 2022. MedMCQA: A large-scale multi-subject multi-choice dataset for medical domain question answering. In *Proceedings of the Conference on Health, Inference, and Learning*. 248–260.

[120] Yu Pan, Jianxin Sun, and Hongfeng Yu. 2023. LM-DiskANN: Low memory footprint in disk-native dynamic graph-based ANN indexing. In *Proceedings of the 2023 IEEE International Conference on Big Data*. 5987–5996.

[121] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: A method for automatic evaluation of machine translation. In *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics*. ACL, 311–318.

[122] Guilherme Penedo, Quentin Malartic, Daniel Hesslow, Ruxandra Cojocaru, Hamza Alobeidli, Alessandro Cappelli, Baptiste Pannier, Ebtesam Almazrouei, and Julien Launay. 2023. The refinedweb dataset for falcon LLM: Outperforming curated corpora with web data only. In *Proceedings of the Advances in Neural Information Processing Systems*. 79155–79172.

[123] Fuchun Peng, X. Huang, Dale Schuurmans, and Nick Cercone. 2002. Investigating the relationship between word segmentation performance and retrieval performance in chinese IR. In *Proceedings of the 19th International Conference on Computational Linguistics*. 1–7.

[124] Fabio Petroni, Aleksandra Piktus, Angela Fan, Patrick Lewis, Majid Yazdani, Nicola De Cao, James Thorne, Yacine Jernite, Vladimir Karpukhin, Jean Maillard, Vassilis Plachouras, Tim Rocktäschel, and Sebastian Riedel. 2021. KILT: A benchmark for knowledge intensive language tasks. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACL, 2523–2544.

[125] Hongjin Qian, Zheng Liu, Peitian Zhang, Kelong Mao, Defu Lian, Zhicheng Dou, and Tiejun Huang. 2025. MemoRAG: Boosting long context processing with global memory-enhanced retrieval augmentation. In *Proceedings of the ACM on Web Conference 2025*. ACM, 2366–2377.

[126] Zhen Qin, R. Jagerman, Kai Hui, Honglei Zhuang, Junru Wu, Jiaming Shen, Tianqi Liu, Jialu Liu, Donald Metzler, Xuanhui Wang, and Michael Bendersky. 2023. Large language models are effective text rankers with pairwise ranking prompting. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics*. 1504–1518.

[127] Derrick Quinn, Mohammad Nouri, Neel Patel, John Salihu, Alireza Salemi, Sukhan Lee, Hamed Zamani, and Mohammad Alian. 2025. Accelerating retrieval-augmented generation. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*. ACM, 15–32.

[128] Colin Raffel, Noam M. Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21 (2020), 140:1–140:67.

[129] Ori Ram, Yoav Levine, Itay Dalmedigos, Dor Muhlgay, Amnon Shashua, Kevin Leyton-Brown, and Yoav Shoham. 2023. In-context retrieval-augmented language models. *Transactions of the Association for Computational Linguistics* 11 (2023), 1316–1331.

[130] Ori Ram, Gal Shachaf, Omer Levy, Jonathan Berant, and Amir Globerson. 2022. Learning to retrieve passages without supervision. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. ACL.

[131] David Rau, Hervé Déjean, Nadezhda Chirkova, Thibault Formal, Shuai Wang, Stéphane Clinchant, and Vassilina Nikoulina. 2024. BERGEN: A benchmarking library for retrieval-augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 7640–7663.

[132] David Rau, Shuai Wang, Hervé Déjean, Stéphane Clinchant, and Jaap Kamps. 2025. Context embeddings for efficient answer generation in retrieval-augmented generation. In *Proceedings of the 18th ACM International Conference on Web Search and Data Mining*. ACM, 493–502.

[133] Nils Reimers and Iryna Gurevych. 2019. Sentence-BERT: Sentence embeddings using siamese BERT-networks. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 3980–3990.

[134] Stephen Robertson and Hugo Zaragoza. 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends® in Information Retrieval* 3, 4 (2009), 333–389.

[135] Jon Saad-Falcon, O. Khattab, Christopher Potts, and Matei Zaharia. 2023. ARES: An automated evaluation framework for retrieval-augmented generation systems. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics*.

[136] Alireza Salemi, Surya Kallumadi, and Hamed Zamani. 2024. Optimization methods for personalizing large language models through retrieval augmentation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 752–762.

[137] Alireza Salemi and Hamed Zamani. 2024. Evaluating retrieval quality in retrieval-augmented generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2395–2400.

[138] Parth Sarthi, Salman Abdullah, Aditi Tuli, Shubh Khanna, Anna Goldie, and Christopher D. Manning. 2024. RAPTOR: Recursive abstractive processing for tree-organized retrieval. In *Proceedings of the International Conference on Learning Representations*. 1–22.

[139] Zhihong Shao, Yeyun Gong, Yelong Shen, Minlie Huang, Nan Duan, and Weizhu Chen. 2023. Enhancing retrieval-augmented large language models with iterative retrieval-generation synergy. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. ACL, 9248–9274.

[140] Teng Shi, Jun Xu, Xiao Zhang, Xiaoxue Zang, Kai Zheng, Yang Song, and Han Li. 2025. Retrieval augmented generation with collaborative filtering for personalized text generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 1294–1304.

[141] Weijia Shi, Sewon Min, Michihiro Yasunaga, Minjoon Seo, Richard James, Mike Lewis, Luke Zettlemoyer, and Wen-tau Yih. 2023. REPLUG: Retrieval-augmented black-box language models. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics*. ACL, 8371–8384.

[142] Yunxiao Shi, Xing Zi, Zijing Shi, Haimin Zhang, Qiang Wu, and Min Xu. 2024. ERAGent: Enhancing retrieval-augmented language models with improved accuracy, efficiency, and personalization. In *Proceedings of the European Conference on Artificial Intelligence*. 2258–2265.

[143] Jinyan Su, Jennifer Healey, Preslav Nakov, and Claire Cardie. 2025. Fast or better? Balancing accuracy and cost in retrieval-augmented generation with flexible user control. arXiv:2502.12145. Retrieved from https://arxiv.org/abs/2502.12145

[144] Weihang Su, Yichen Tang, Qingyao Ai, Junxi Yan, Changyue Wang, Hongning Wang, Ziyi Ye, Yujia Zhou, and Yiqun Liu. 2025. Parametric retrieval augmented generation. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 1240–1250.

[145] Weihang Su, Yichen Tang, Qingyao Ai, Zhijing Wu, and Yiqun Liu. 2024. DRAGIN: Dynamic retrieval augmented generation based on the real-time information needs of large language models. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics*. 12991–13013.

[146] Hao Sun, Hengyi Cai, Yuchen Li, Xuanbo Fan, Xiaochi Wei, Shuaiqiang Wang, Yan Zhang, and Dawei Yin. 2025. Enhancing retrieval-augmented generation via evidence tree search. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 24116–24127.

[147] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? Investigating large language models as re-ranking agents. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 14918–14937.

[148] Zhiqing Sun, Xuezhi Wang, Yi Tay, Yiming Yang, and Denny Zhou. 2023. Recitation-augmented language models. In *Proceedings of the International Conference on Learning Representations*. 1–23.

[149] Kento Tatsuno, Daisuke Miyashita, Taiga Ikeda, Kiyoshi Ishiyama, Kazunari Sumiyoshi, and Jun Deguchi. 2024. AiSAQ: All-in-storage ANNS with product quantization for DRAM-free information retrieval. arXiv:2404.06004. Retrieved from https://arxiv.org/abs/2404.06004

[150] Yi Tay, Mostafa Dehghani, Vinh Q. Tran, Xavier Garcia, Jason Wei, Xuezhi Wang, Hyung Won Chung, Siamak Shakeri, Dara Bahri, Tal Schuster, Huaixiu Steven Zheng, Denny Zhou, Neil Houlsby, and Donald Metzler. 2023. UL2: Unifying language learning paradigms. In *Proceedings of the International Conference on Learning Representations*. 1–33.

[151] Nandan Thakur, Luiz Bonifacio, Crystina Zhang, Odunayo Ogundepo, Ehsan Kamalloo, David Alfonso-Hermelo, Xiaoguang Li, Qun Liu, Boxing Chen, Mehdi Rezagholizadeh, and Jimmy Lin. 2024. "Knowing when you don't Know": A multilingual relevance assessment dataset for robust Retrieval-Augmented Generation. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. ACL, 12508–12526.

[152] James Thorne, Andreas Vlachos, Christos Christodoulopoulos, and Arpit Mittal. 2018. FEVER: A large-scale dataset for fact extraction and verification. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*. ACL, 809–819.

[153] Hugo Touvron, Louis Martin, Kevin R. Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Niko-lay Bashlykov, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. arXiv:2307.09288. Retrieved from https://arxiv.org/abs/2307.09288

[154] Giovanni Trappolini, Florin Cuconasu, Simone Filice, Y. Maarek, and Fabrizio Silvestri. 2026. Redefining retrieval evaluation in the era of LLMs. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 1–14.

[155] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2023. Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 10014–10037.

[156] George Tsatsaronis, Georgios Balikas, Prodromos Malakasiotis, Ioannis Partalas, Matthias Zschunke, Michael R. Alvers, Dirk Weissenborn, Anastasia Krithara, Sergios Petridis, Dimitris Polychronopoulos, Yannis Almirantis, John Pavlopoulos, Nicolas Baskiotis, Patrick Gallinari, Thierry Artiéres, Axel-Cyrille Ngonga Ngomo, Norman Heino, Eric Gaussier, Liliana Barrio-Alvers, Michael Schroeder, Ion Androutsopoulos, and Georgios Paliouras. 2015. An overview of the BIOASQ large-scale biomedical semantic indexing and question answering competition. *BMC Bioinformatics* 16, 1 (2015), 138.

[157] Ashish Vaswani, Noam M. Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Proceedings of the Advances in Neural Information Processing Systems* 30 (2017), 6000–6010.

[158] Aditi Vidyarthi, Khoirom Motilal Singh, and Dennis Singh Moirangthem. 2026. SageRAG: Query rewriting for retrieval enhancement and retrieval-augmented generation for grounded responses in AI research Assistance. *Expert Systems with Applications* 308 (2026), 131160.

[159] Alex Wang, Yada Pruksachatkun, Nikita Nangia, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel R. Bowman. 2019. SuperGLUE: A stickier benchmark for general-purpose language understanding systems. In *Proceedings of the Advances in Neural Information Processing Systems* 32 (2019), 3266–3280.

[160] Haoyu Wang, Ruirui Li, Haoming Jiang, Jinjin Tian, Zhengyang Wang, Chen Luo, Xianfeng Tang, Monica Xiao Cheng, Tuo Zhao, and Jing Gao. 2024. BlendFilter: Advancing retrieval-augmented large language models via query generation blending and knowledge filtering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 1009–1025.

[161] Liang Wang, Nan Yang, Xiaolong Huang, Binxing Jiao, Linjun Yang, Daxin Jiang, Rangan Majumder, and Furu Wei. 2022. Text embeddings by weakly-supervised contrastive pre-training. arXiv:2212.03533. Retrieved from https://arxiv.org/abs/2212.03533

[162] Liang Wang, Nan Yang, and Furu Wei. 2023. Query2doc: Query expansion with large language models. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 9414–9423.

[163] Qifan Wang, Yi Fang, Anirudh Ravula, Fuli Feng, Xiaojun Quan, and Dongfang Liu. 2022. WebFormer: The web-page transformer for structure information extraction. In *Proceedings of the ACM Web Conference 2022*. 3124–3133.

[164] Xiaohua Wang, Zhenghua Wang, Xuan Gao, Feiran Zhang, Yixin Wu, Zhibo Xu, Tianyuan Shi, Zhengyuan Wang, Shizheng Li, Qi Qian, et al. 2024. Searching for best practices in retrieval-augmented generation. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. 17716–17736.

[165] Xintao Wang, Qianwen Yang, Yongting Qiu, Jiaqing Liang, Qianyu He, Zhouhong Gu, Yanghua Xiao, and Wei Wang. 2023. KnowledGPT: Enhancing large language models with retrieval and storage access on knowledge bases. arXiv:2308.11761. Retrieved from https://arxiv.org/abs/2308.11761

[166] Zhiruo Wang, Jun Araki, Zhengbao Jiang, Md. Rizwan Parvez, and Graham Neubig. 2023. Learning to filter context for retrieval-augmented generation. arXiv:2311.08377. Retrieved from https://arxiv.org/abs/2311.08377

[167] BigScience Workshop: Teven Le Scao, Angela Fan, Christopher Akiki, Ellie Pavlick, Suzana Ilić, Daniel Hesslow, Roman Castagné, Alexandra Sasha Luccioni, François Yvon, Matthias Gallé, et al. 2022. BLOOM: A 176B-parameter open-access multilingual language model. arXiv:2211.05100. Retrieved from https://arxiv.org/abs/2211.05100

[168] Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. 2024. C-Pack: Packed resources for general chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 641–649.

[169] Guangzhi Xiong, Qiao Jin, Zhiyong Lu, and Aidong Zhang. 2024. Benchmarking retrieval-augmented generation for medicine. In *Findings of the Association for Computational Linguistics: ACL 2024*. ACL, 6233–6251.

[170] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate nearest neighbor negative contrastive learning for dense text retrieval. In *Proceedings of the International Conference on Learning Representations*. 1–16.

[171] Fangyuan Xu, Weijia Shi, and Eunsol Choi. 2024. RECOMP: Improving retrieval-augmented LMs with context compression and selective augmentation. In *Proceedings of the International Conference on Learning Representations*. 1–25.

[172] Shicheng Xu, Liang Pang, Jun Xu, Huawei Shen, and Xueqi Cheng. 2024. List-aware reranking-truncation joint model for search and retrieval-augmented generation. In *Proceedings of the ACM on Web Conference 2024*. ACM, 1330–1340.

[173] Shi-Qi Yan, Jia-Chen Gu, Yun Zhu, and Zhen-Hua Ling. 2024. Corrective retrieval augmented generation. arXiv:2401.15884. Retrieved from https://arxiv.org/abs/2401.15884

[174] Diji Yang, Jinmeng Rao, Kezhen Chen, Xiaoyuan Guo, Yawen Zhang, Jie Yang, and Yi Zhang. 2024. IM-RAG: Multi-round retrieval-augmented generation through learning inner monologues. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 730–740.

[175] Haoyan Yang, Zhitao Li, Yong Zhang, Jianzong Wang, Ning Cheng, Ming Li, and Jing Xiao. 2023. PRCA: Fitting black-box large language models for retrieval question answering via pluggable reward-driven contextual adapter. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 5364–5375.

[176] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the Conference on Empirical Methods in Natural Language Processing*. ACL, 2369–2380.

[177] Mert Yazan, Suzan Verberne, and Frederik Situmeang. 2025. Improving RAG for personalization with author features and contrastive examples. In *Proceedings of the European Conference on Information Retrieval*. 408–416.

[178] Zheng Ye, Jimmy X. Huang, and Hongfei Lin. 2011. Finding a good query-related topic for boosting pseudo-relevance feedback. *Journal of the American Society for Information Science and Technology* 62, 4 (2011), 748–760.

[179] Hao Yu, Aoran Gan, Kai Zhang, Shiwei Tong, Qi Liu, and Zhaofeng Liu. 2025. Evaluation of retrieval-augmented generation: A survey. In *Proceedings of the Big Data*. Springer Nature Singapore, 102–120.

[180] Shi Yu, Jiahua Liu, Jingqin Yang, Chenyan Xiong, Paul N. Bennett, Jianfeng Gao, and Zhiyuan Liu. 2020. Few-shot generative conversational query rewriting. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 1933–1936.

[181] Tan Yu, Anbang Xu, and Rama Akkiraju. 2024. In defense of RAG in the era of long-context language models. arXiv:2409.01666. Retrieved from https://arxiv.org/abs/2409.01666

[182] Wenhao Yu, Dan Iter, Shuohang Wang, Yichong Xu, Mingxuan Ju, Soumya Sanyal, Chenguang Zhu, Michael Zeng, and Meng Jiang. 2023. Generate rather than retrieve: Large language models are strong context generators. In *Proceedings of the International Conference on Learning Representations*. 1–27.

[183] Zichun Yu, Chenyan Xiong, Shi Yu, and Zhiyuan Liu. 2023. Augmentation-adapted retriever improves generalization of language models as generic plug-in. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. ACL, 2421–2436.

[184] Zhenrui Yue, Huimin Zeng, Yimeng Lu, Lanyu Shang, Yang Zhang, and Dong Wang. 2024. Evidence-driven retrieval augmented response generation for online misinformation. In *Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics*. 5628–5643.

[185] Saber Zerhoudi and Michael Granitzer. 2024. PersonaRAG: Enhancing retrieval-augmented generation systems with user-centric agents. In *Proceedings of the Workshop Information Retrieval's Role in RAG Systems co-located with the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1–11.

[186] Susan Zhang, Stephen Roller, Naman Goyal, Mikel Artetxe, Moya Chen, Shuohui Chen, Christopher Dewan, Mona T. Diab, Xian Li, Xi Victoria Lin, et al. 2022. OPT: Open pre-trained transformer language models. arXiv:2205.01068. Retrieved from https://arxiv.org/abs/2205.01068

[187] Penghao Zhao, Hailin Zhang, Qinhan Yu, Zhengren Wang, Yunteng Geng, Fangcheng Fu, Ling Yang, Wentao Zhang, Jie Jiang, and Bin Cui. 2026. Retrieval-augmented generation for AI-generated content: A survey. *Data Science and Engineering* 11, 1 (2026), 1–29.

[188] Wayne Xin Zhao, Jing Liu, Ruiyang Ren, and Ji-Rong Wen. 2024. Dense text retrieval based on pretrained language models: A survey. *ACM Transactions on Information Systems* 42, 4 (2024), 1–60.

[189] Huaixiu Steven Zheng, Swaroop Mishra, Xinyun Chen, Heng-Tze Cheng, Ed H. Chi, Quoc V. Le, and Denny Zhou. 2024. Take a step back: Evoking reasoning via abstraction in large language models. In *Proceedings of the International Conference on Learning Representations*. 1–38.

[190] Ning Zhong, Yuefeng Li, and Sheng-Tang Wu. 2012. Effective pattern discovery for text mining. *IEEE Transactions on Knowledge and Data Engineering* 24, 1 (2012), 30–44.

[191] Yun Zhu, Jia-Chen Gu, Caitlin Sikora, Ho Ko, Yinxiao Liu, Chu-Cheng Lin, Lei Shu, Liangchen Luo, Lei Meng, Bang Liu, and Jindong Chen. 2025. Accelerating inference of retrieval-augmented generation via sparse context selection. In *Proceedings of the International Conference on Learning Representations*. 1–17.

[192] Yutao Zhu, Huaying Yuan, Shuting Wang, Jiongnan Liu, Wenhan Liu, Chenlong Deng, Haonan Chen, Zheng Liu, Zhicheng Dou, and Ji-Rong Wen. 2025. Large language models for information retrieval: A survey. *ACM Transactions on Information Systems* 44, 1 (2025), 1–54.

[193] Honglei Zhuang, Zhen Qin, Rolf Jagerman, Kai Hui, Ji Ma, Jing Lu, Jianmo Ni, Xuanhui Wang, and Michael Bendersky. 2023. RankT5: Fine-tuning T5 for text ranking with ranking losses. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, 2308–2313.

Received 22 August 2024; revised 21 February 2026; accepted 18 March 2026